

C. Attention and Transformers

C.1 Attention

C.2 Transformer

We now specialize the general attention operator into two key forms, Multi-head attention and Self-attention. Then we will see that stacking these with *feed-forward layers*, *residual connections*, and *normalization* gives the Transformer architecture.



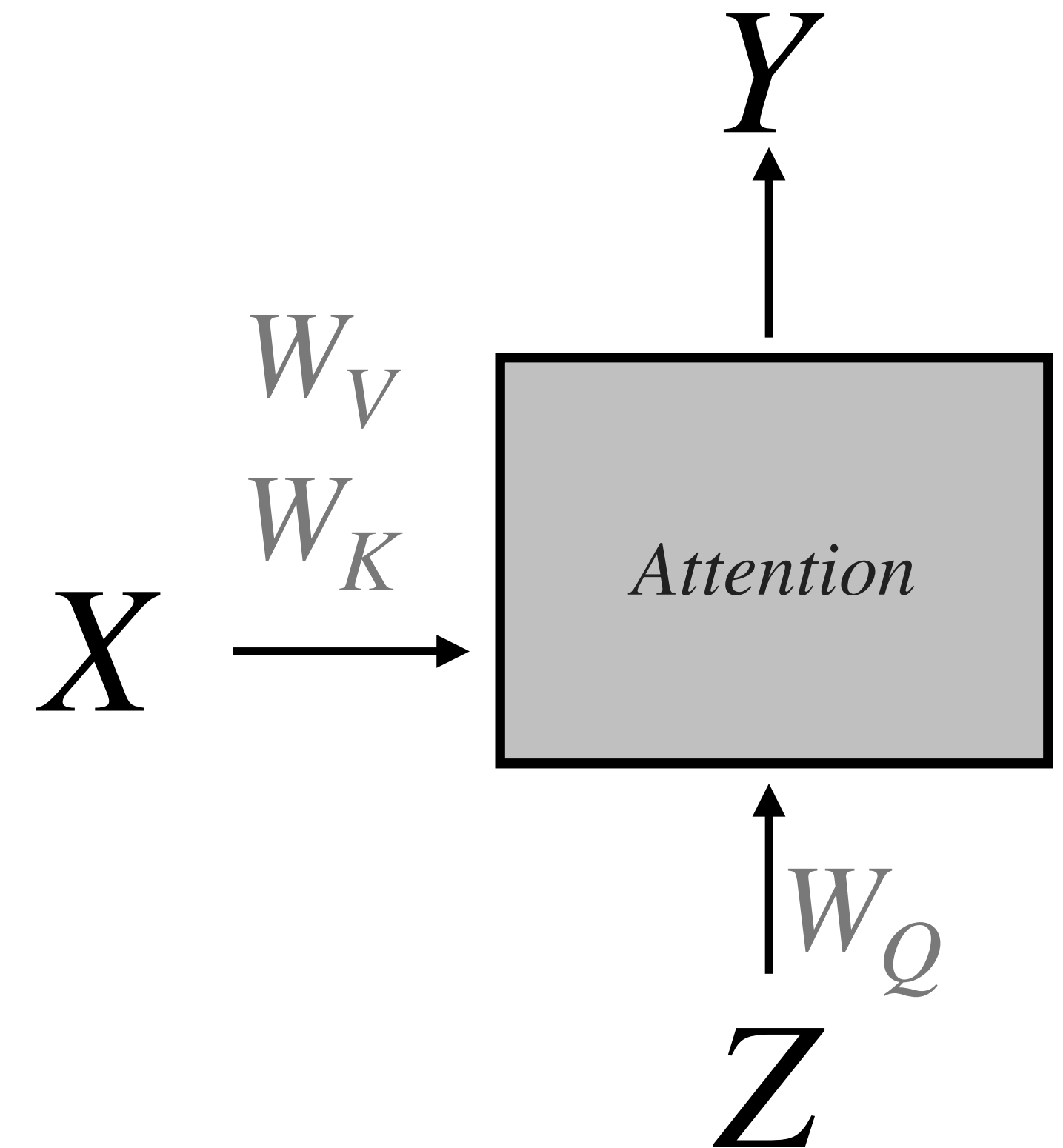
Tiam Heydari, PhD
Postdoctoral Fellow, UBC

1) Multi-head attention

In part II of this lecture, we have seen that attention operator allows each element of $\mathbf{Z}$ to softly reads from all elements of $\mathbf{X}$ and update itself. And result is called $\mathbf{Y}$

$$\begin{aligned}
 \mathbf{X} &= \begin{bmatrix} x_1^\top \\ \vdots \\ x_T^\top \end{bmatrix} & \mathbf{Z} &= \begin{bmatrix} z_1^\top \\ \vdots \\ z_{T'}^\top \end{bmatrix} & \mathbf{Q} &= \mathbf{Z}\mathbf{W}_Q & \mathbf{W}_Q &\in \mathbb{R}^{d_z \times d_k} \\
 & & & & \mathbf{K} &= \mathbf{X}\mathbf{W}_K & \mathbf{W}_K &\in \mathbb{R}^{d_x \times d_k} \\
 & & & & \mathbf{V} &= \mathbf{X}\mathbf{W}_V & \mathbf{W}_V &\in \mathbb{R}^{d_x \times d_v}
 \end{aligned}$$

$$\mathbf{Y} = \text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}_{\text{rowwise}} \left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} \right) \mathbf{V}$$

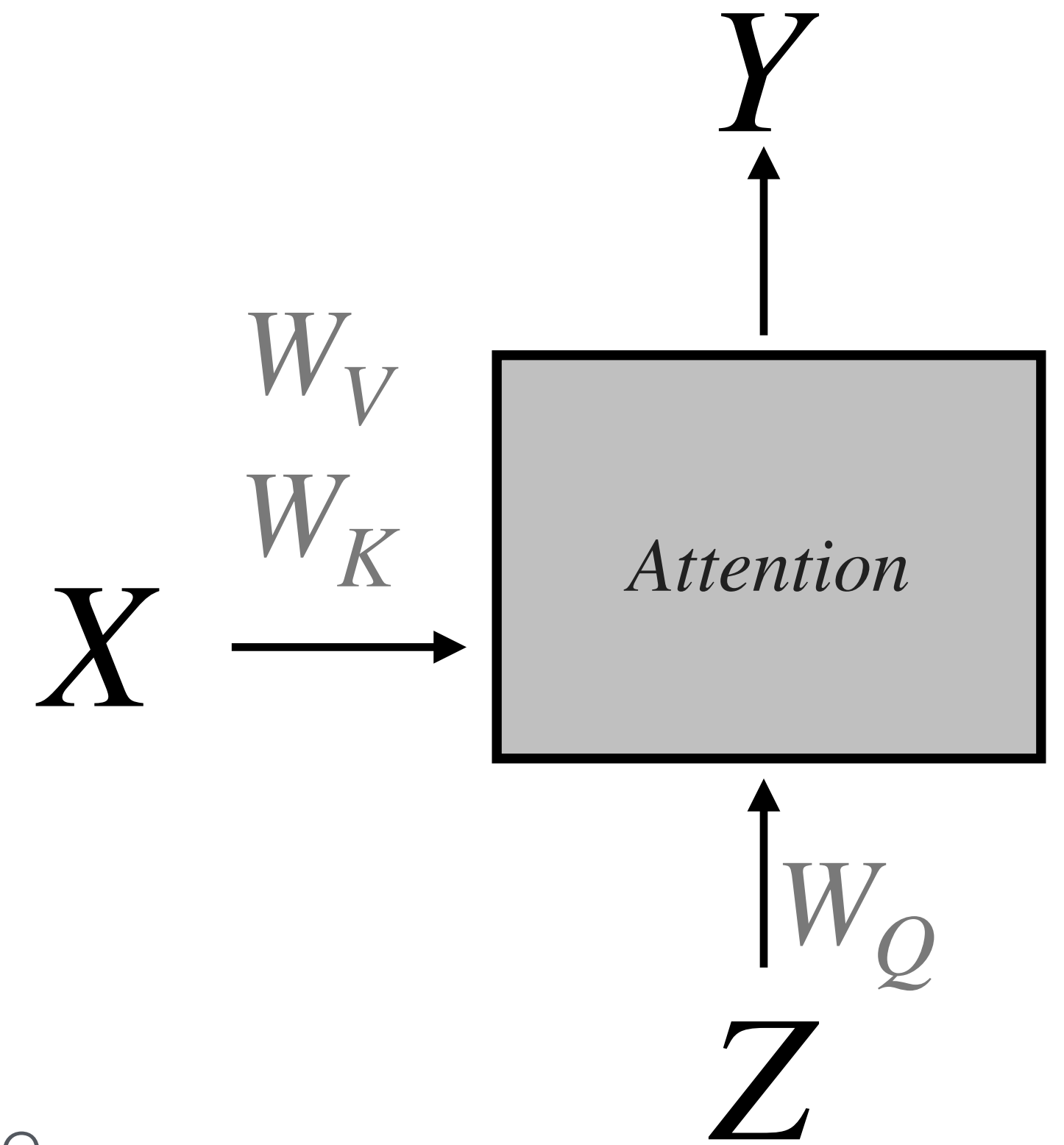


Motivation for multi-head Attention

- one **attention block** produces one specific mixing pattern over the values to produce output Y
- this is useful, but maybe too restrictive and not very stable

In other words:

- A single attention head gives one similarity function and one weighted mixture
- But the same input may contain different kinds of relations at once
- So instead of one attention, we want to run several (k) attentions *in parallel*
- Let each head learn its own W_Q, W_K, W_V



Defining one attention head, h :

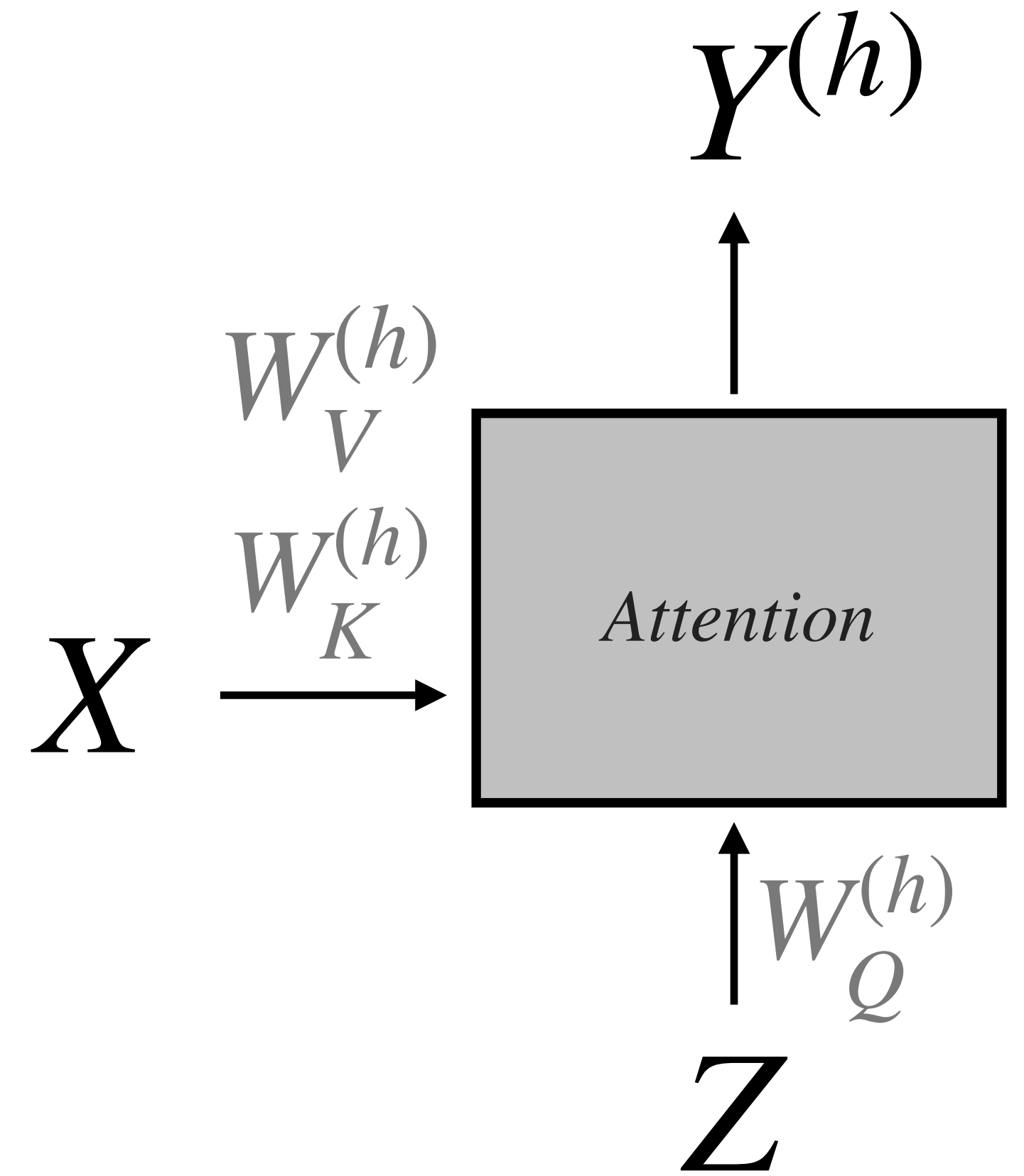
$$X = \begin{bmatrix} x_1^\top \\ \vdots \\ x_T^\top \end{bmatrix} \quad Z = \begin{bmatrix} z_1^\top \\ \vdots \\ z_{T'}^\top \end{bmatrix}$$

$$Q^{(h)} = ZW_Q^{(h)},$$

$$K^{(h)} = XW_K^{(h)}$$

$$V^{(h)} = XW_V^{(h)}$$

$$Y^{(h)} = \text{Attention}(Q^{(h)}, K^{(h)}, V^{(h)})$$

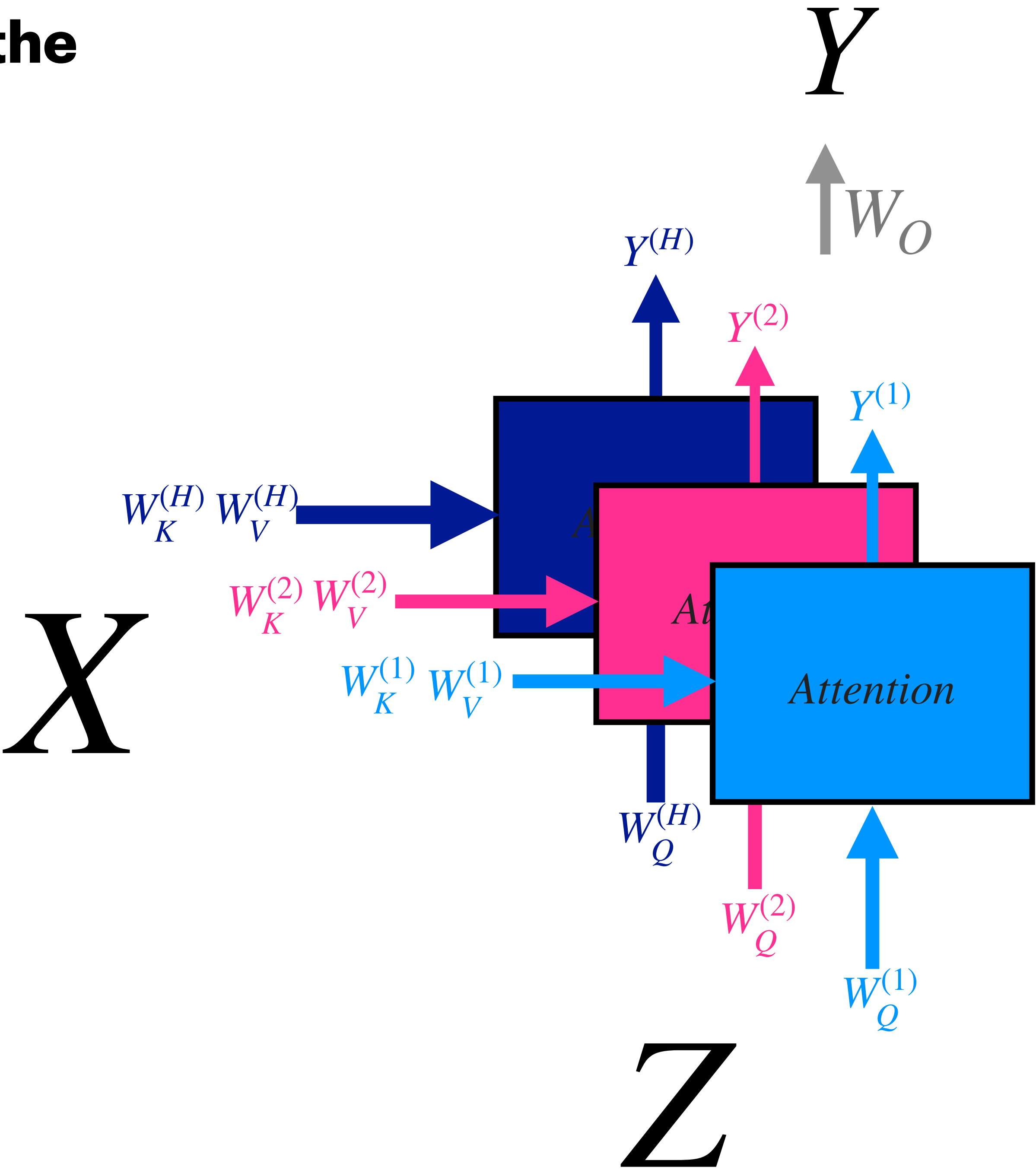


Now we can have multiple heads at the same time

$$X = \begin{bmatrix} x_1^\top \\ \vdots \\ x_T^\top \end{bmatrix} \quad Z = \begin{bmatrix} z_1^\top \\ \vdots \\ z_{T'}^\top \end{bmatrix} \quad \begin{aligned} Q^{(h)} &= ZW_Q^{(h)}, \\ K^{(h)} &= XW_K^{(h)}, \\ V^{(h)} &= XW_V^{(h)} \end{aligned}$$

$$Y^{(h)} = \text{Attention}(Q^{(h)}, K^{(h)}, V^{(h)})$$

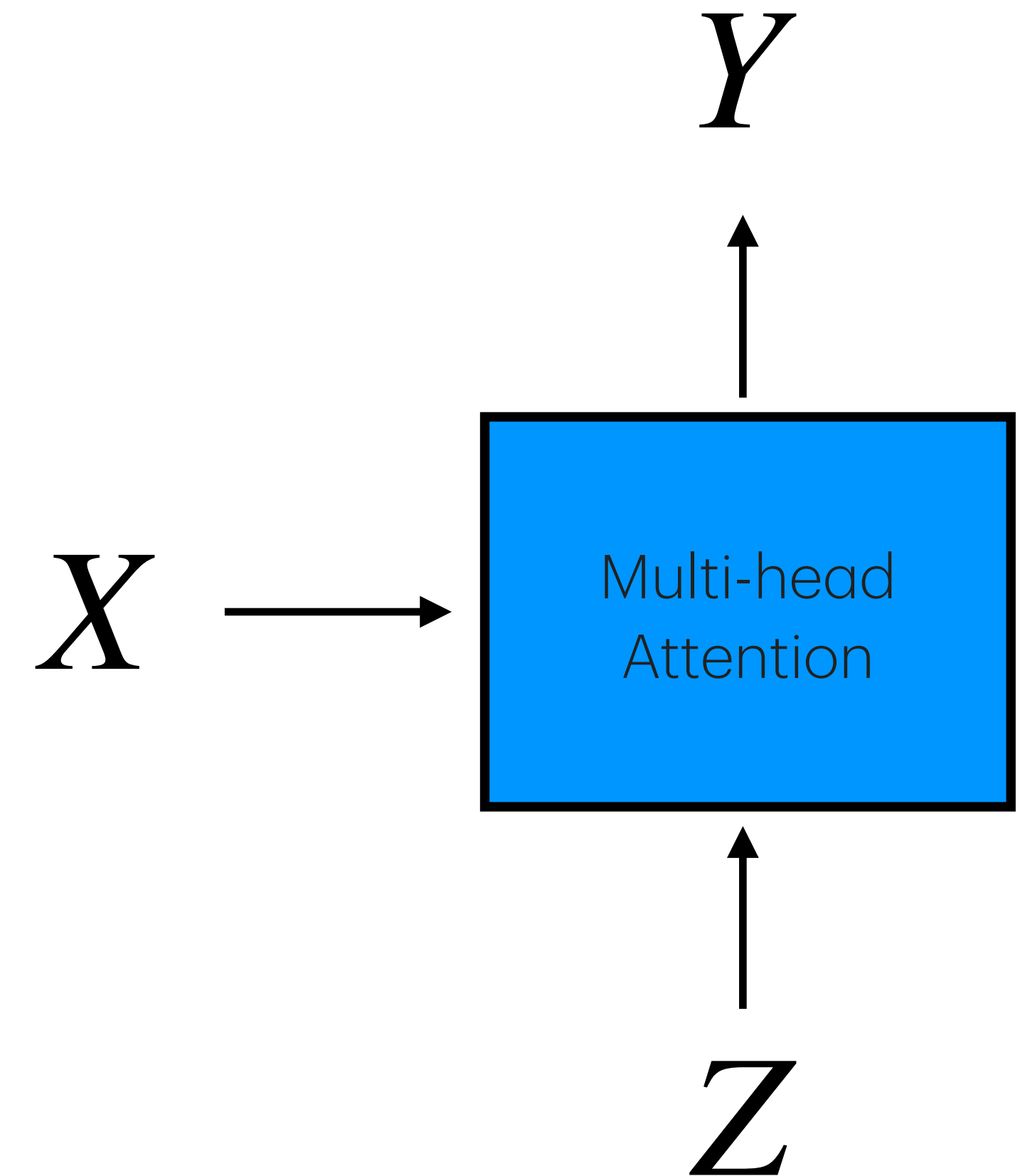
$$Y = \text{Concat}(Y^{(1)}, Y^{(2)}, \dots, Y^{(H)}) W_O$$



$$X = \begin{bmatrix} x_1^\top \\ \vdots \\ x_T^\top \end{bmatrix} \quad Z = \begin{bmatrix} z_1^\top \\ \vdots \\ z_{T'}^\top \end{bmatrix} \quad \begin{aligned} Q^{(h)} &= ZW_Q^{(h)}, \\ K^{(h)} &= XW_K^{(h)}, \\ V^{(h)} &= XW_V^{(h)} \end{aligned}$$

$$Y^{(h)} = \text{Attention}(Q^{(h)}, K^{(h)}, V^{(h)})$$

$$Y = \text{Concat}(Y^{(1)}, Y^{(2)}, \dots, Y^{(H)}) W_O$$



Multi-head attention simply replaces one attention operator with several attention operators in parallel, each with its own softmax weighting pattern.

2) Self-attention

from general attention to self-attention

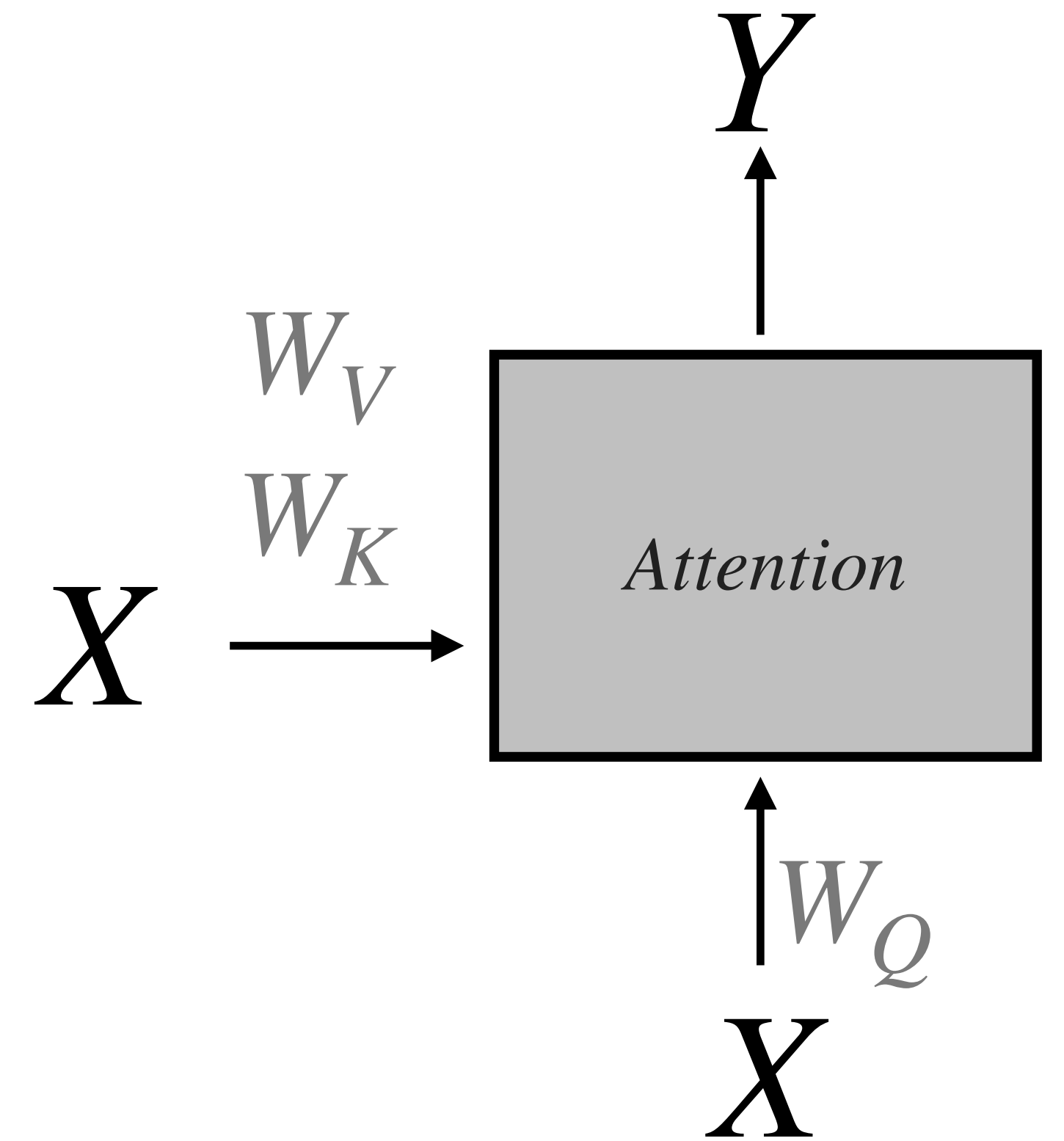
- In general attention, Queries come from Z , while Keys/Values come from X
- In self-attention, these come from the same set of representations So we set:

$$Z = X$$

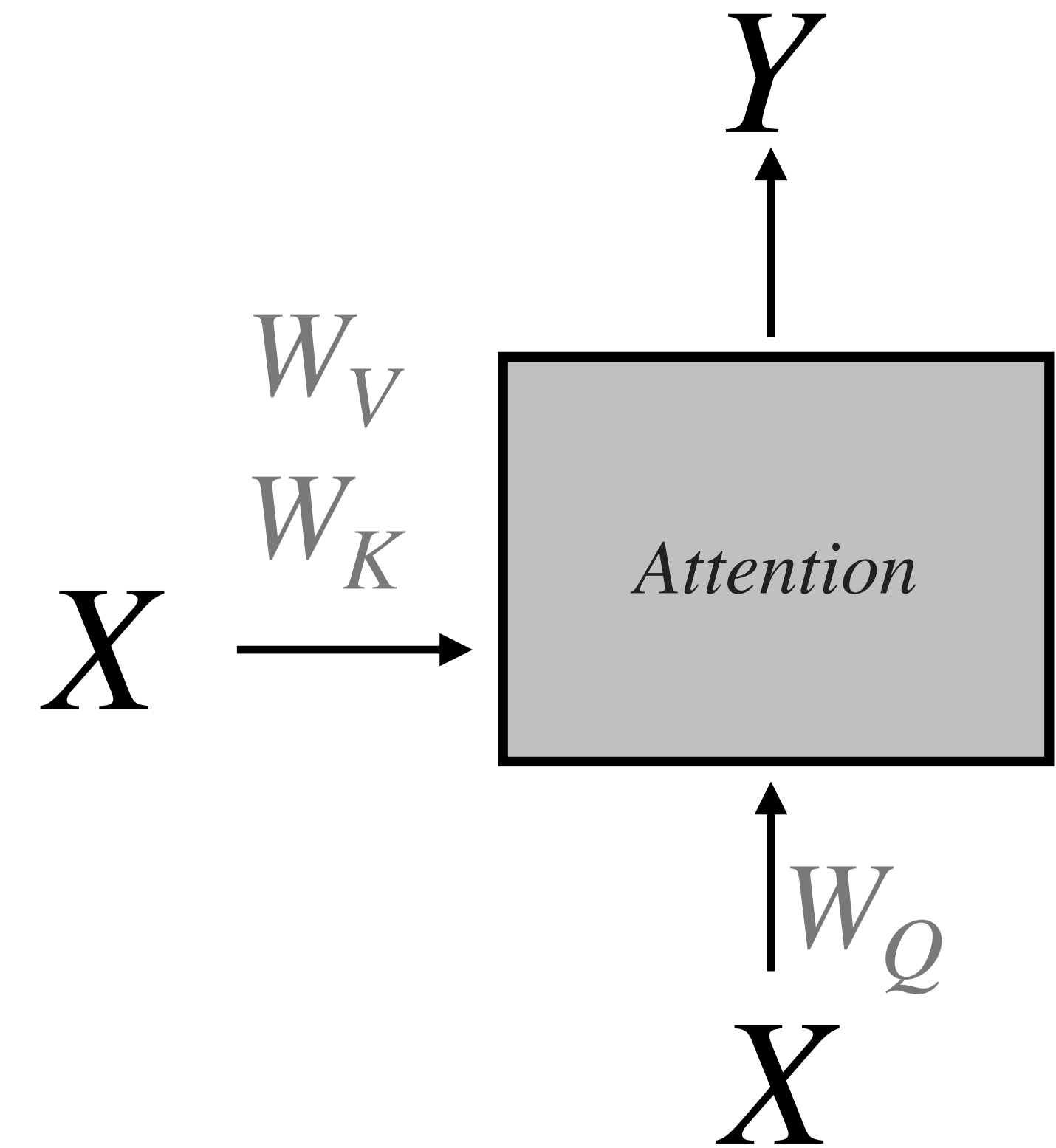
- Self-attention in matrix form:

$$X = \begin{bmatrix} x_1^\top \\ \vdots \\ x_T^\top \end{bmatrix} \quad \begin{aligned} Q &= XW_Q \\ K &= XW_K \\ V &= XW_V \end{aligned} \quad \begin{aligned} W_Q &\in \mathbb{R}^{d_z \times d_k} \\ W_K &\in \mathbb{R}^{d_x \times d_k} \\ W_V &\in \mathbb{R}^{d_x \times d_v} \end{aligned}$$

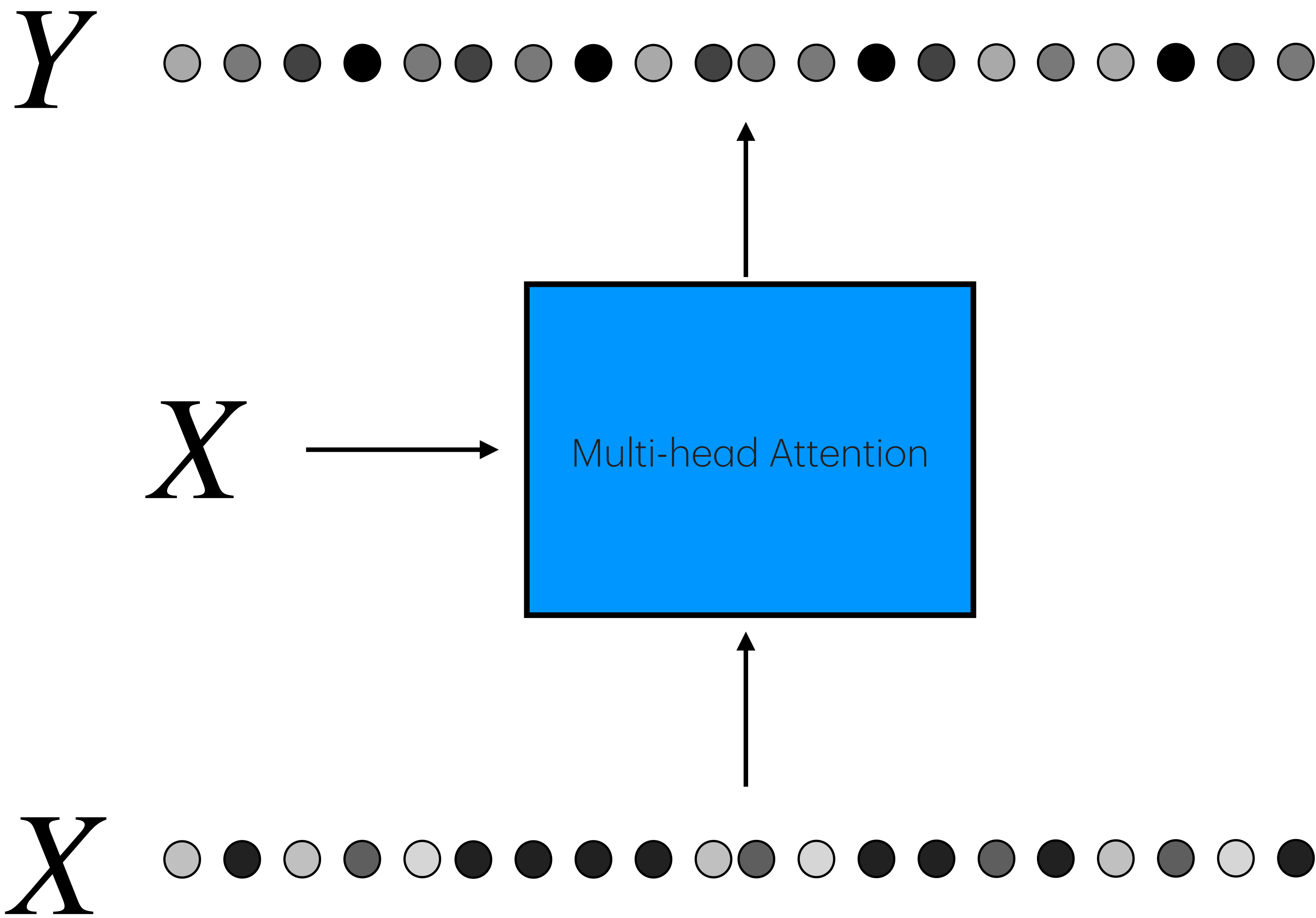
$$Y = \text{Attention}(Q, K, V) = \text{softmax}_{\text{rowwise}} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V$$



- What does self-attention do?
- Each element of X becomes a Query, a Key, and a Value
- So each element can read from all other elements in the same set
- This updates every element using context from the whole set

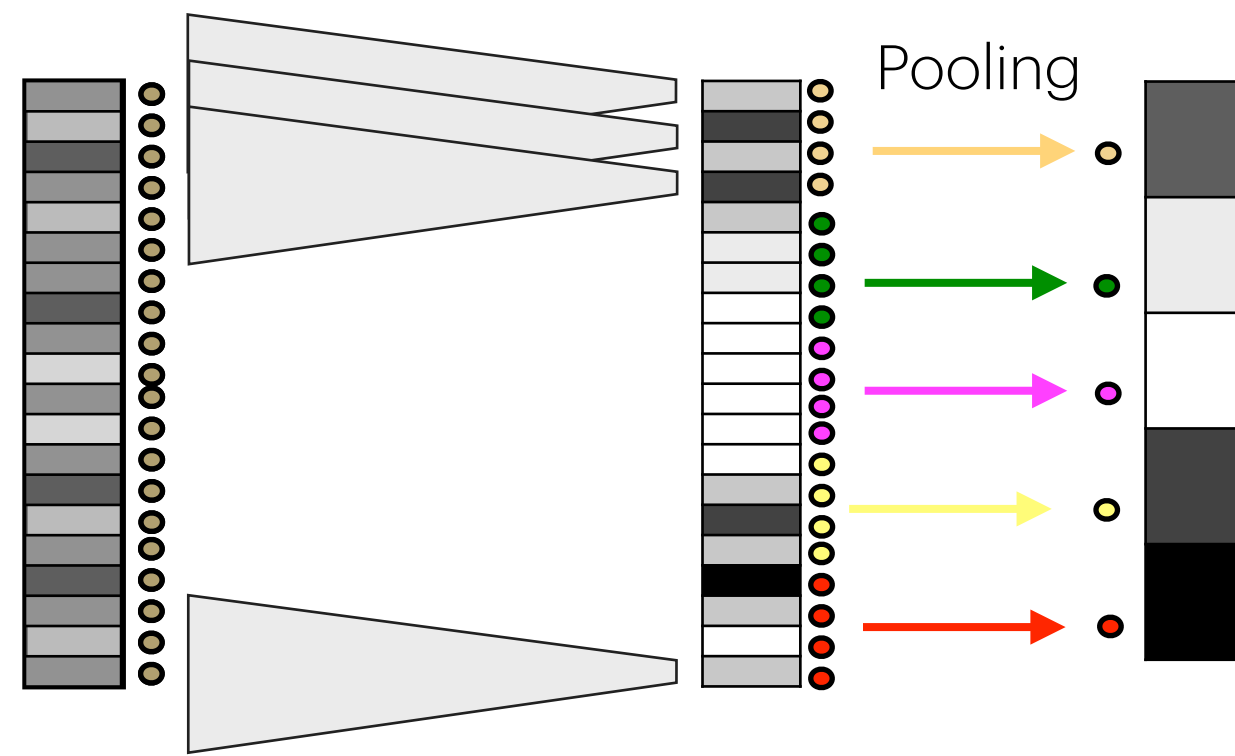


Self-attention lets each element of X update itself by softly reading from all elements, including itself.

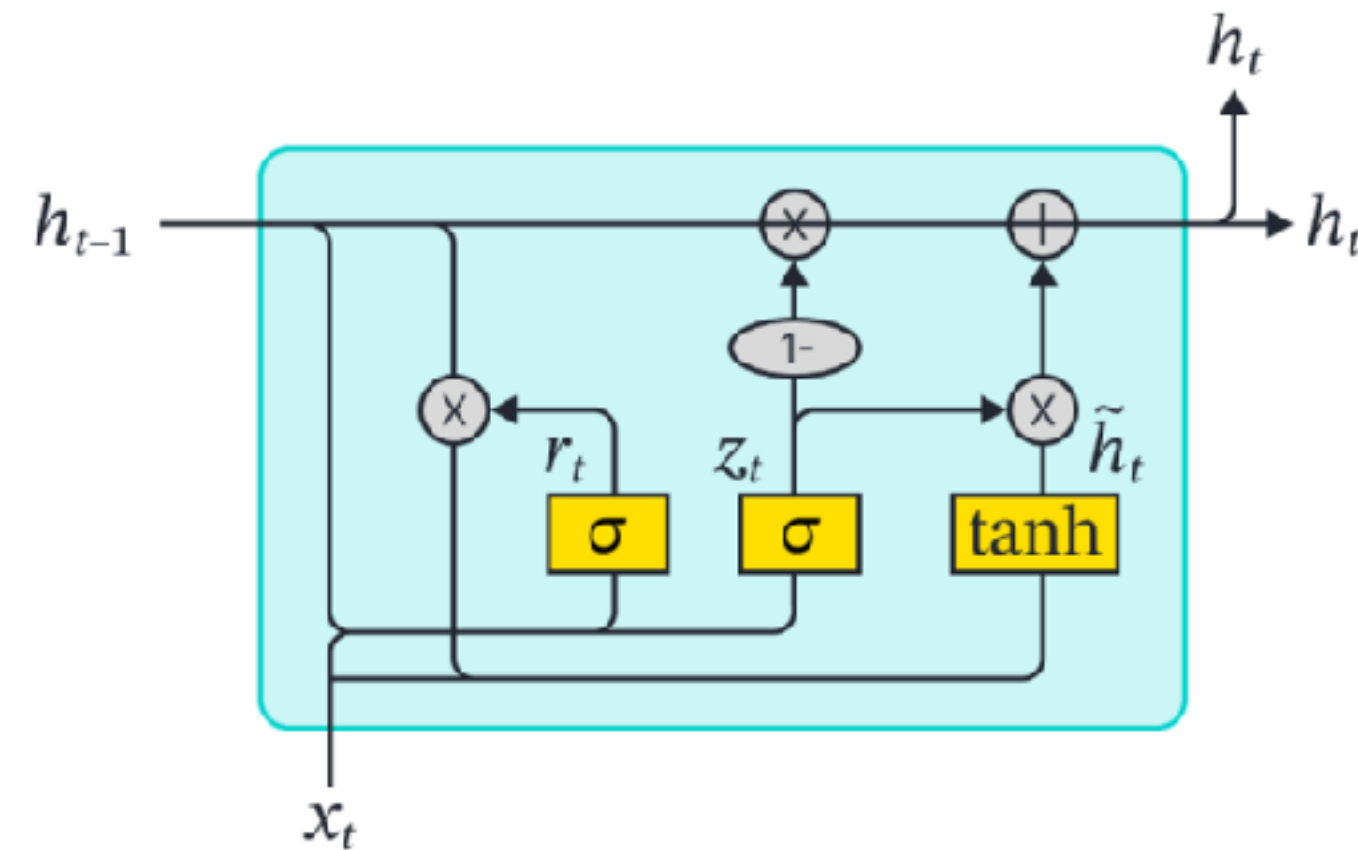


Let's put attention in a more general context, and compare it with some other operators:

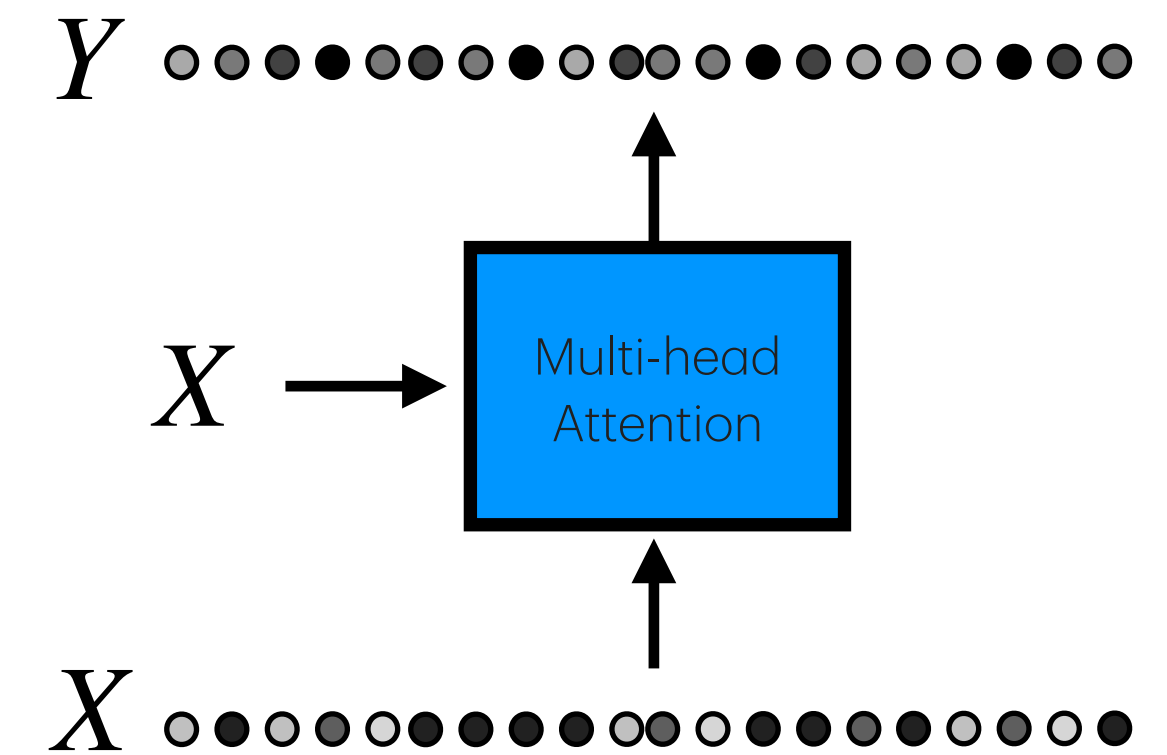
Convolutional Block



Recurrent Networks



Self-Attention



- **Best natural domain:** grids and locally ordered domains
- **Built-in bias:** locality + weight sharing
- **Symmetry view:** translation equivariant
- **Strength:** efficient, parallel computing, strong local inductive bias
- **Weakness:** long-range interactions need many layers / large receptive field

- **Best natural domain:** ordered sequences
- **Built-in bias:** sequential update, directionality / causality
- **Symmetry view:** order matters explicitly
- **Strength:** natural for autoregressive sequence processing
- **Weakness:** hard to parallelize across time, information bottleneck

- **Best natural domain:** sets of vectors; also sequences or grids once positional structure is added.
- **Built-in bias:** content-based interaction between any pair
- **Symmetry view:** permutation equivariant (without positional encoding)
- **Strength:** direct long-range interactions, **high parallelism**
- **Weakness:** dense attention is expensive for long inputs, since it builds the full pairwise score matrix

3) Transformer Architecture

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

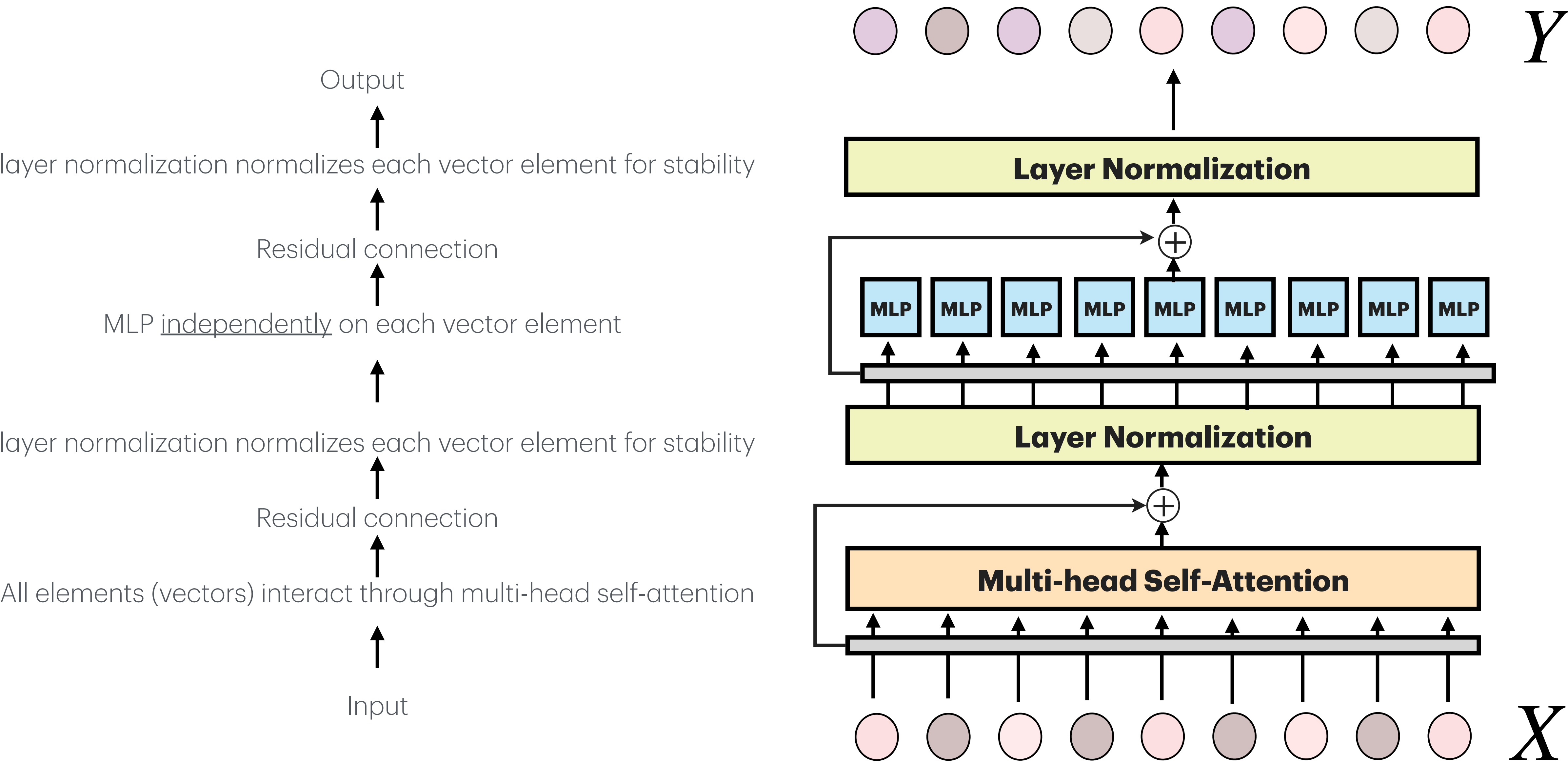
Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

- it is hard to make processors faster, but easy to buy more GPUs. Attention is all you need...(?!)(!)

Transformer is an architecture that puts Self-Attention as the core! Very simple

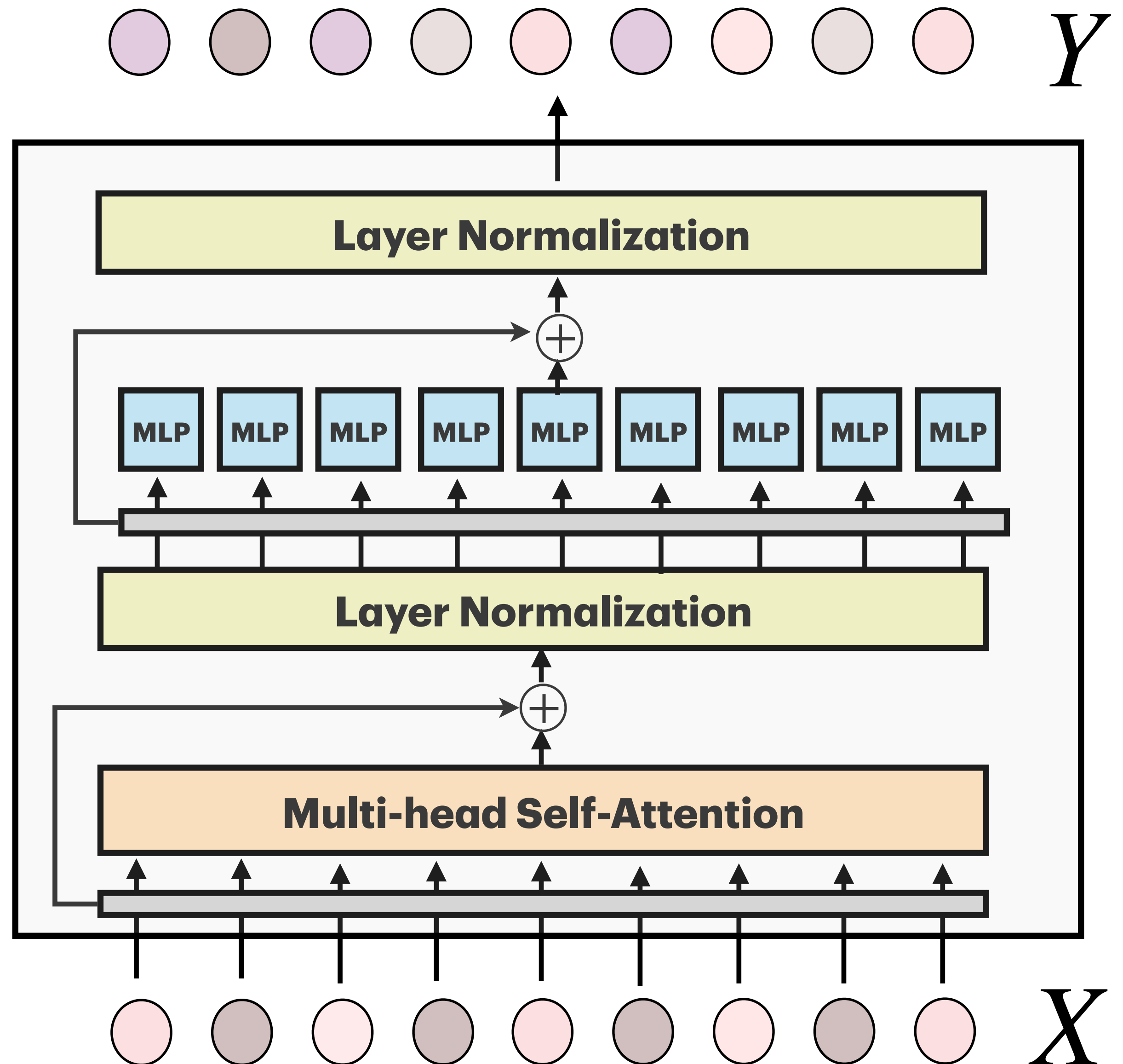


A Transformer Block:

Input: set/sequence X

Output: updated set/sequence Y

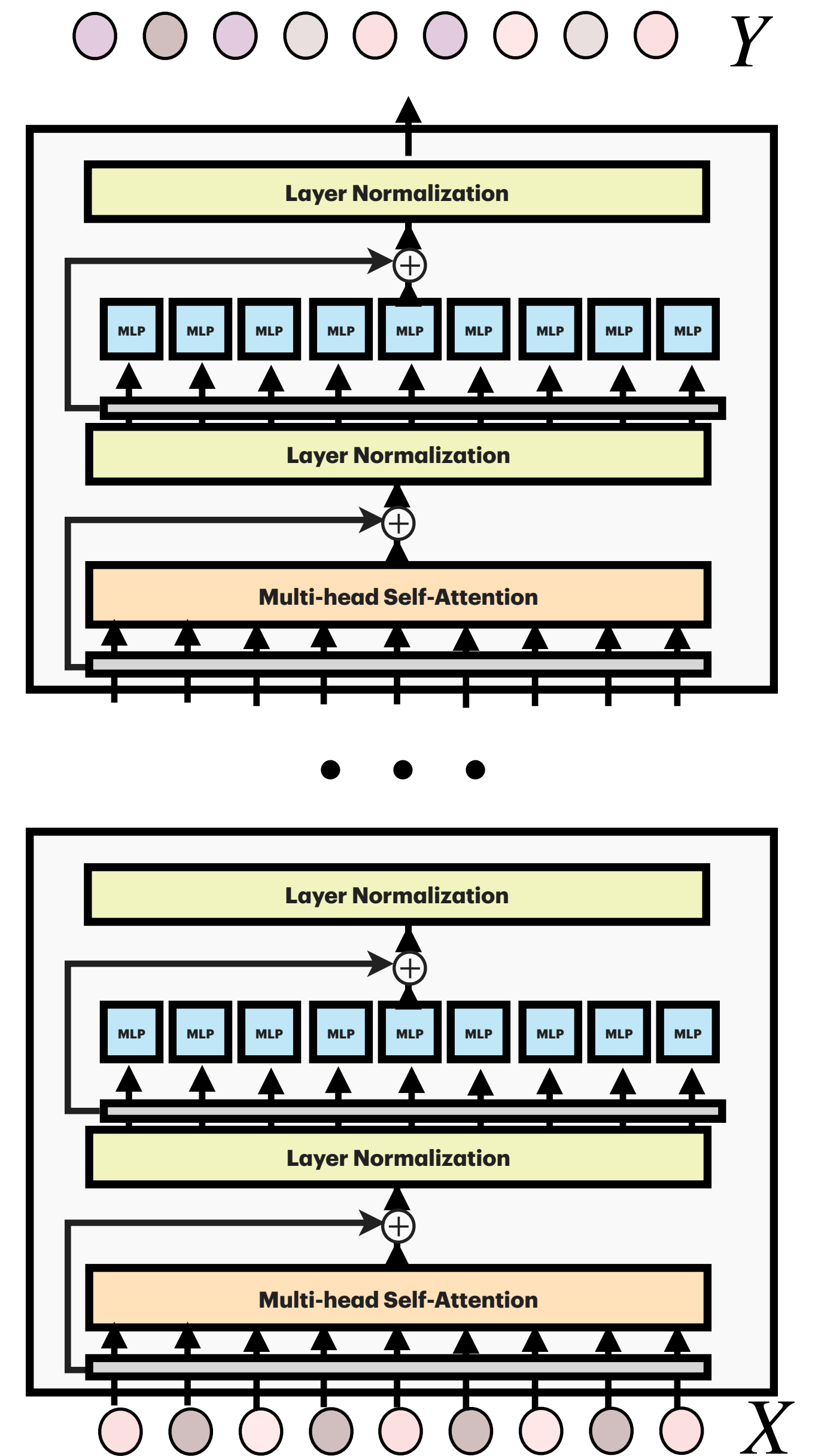
- Multi-head self-attention: the only step in the block where different tokens directly interact with each other
- LayerNorm and MLP: operate on each token independently, with shared weights across positions
- Residual connections: preserve information and help optimization
- Highly parallelizable: most of the computation is large matrix multiplications!



A Transformer is a sequence of Transformer blocks:

Input: set/sequence X

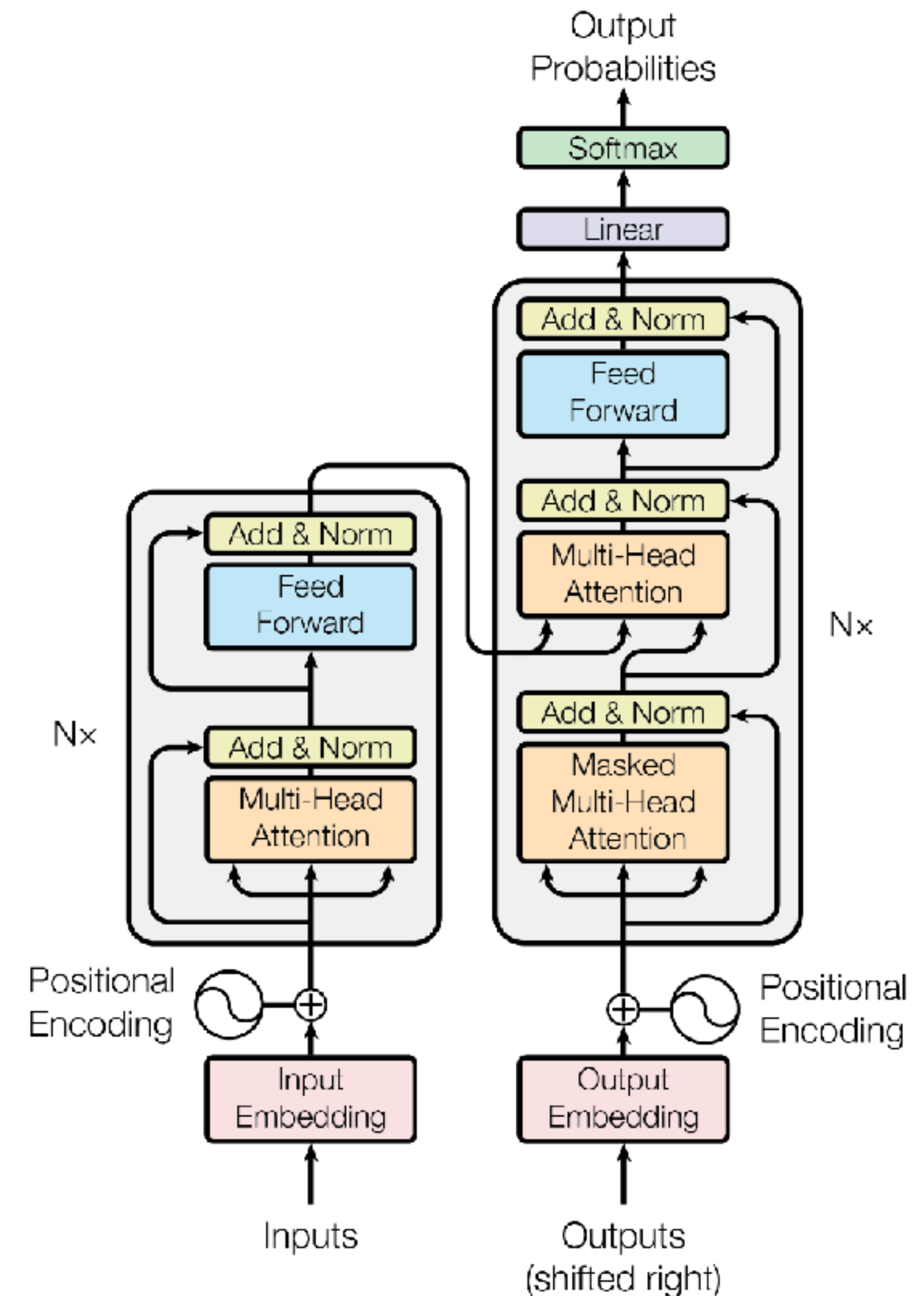
Output: updated set/sequence Y



- Transformers have not changed **much**** since 2017
- They have just become much larger!
- Original Transformer (Vaswani et al., 2017) 12 blocks, (D=1024), (H=16), (N=512) 213M parameters
- GPT-2 (Radford et al., 2019) 48 blocks, (D=1600), (H=25), (N=1024) 1.5B parameters
- GPT-3 (Brown et al., 2020) 96 blocks, (D=12288), (H=96), (N=2048) 175B parameters

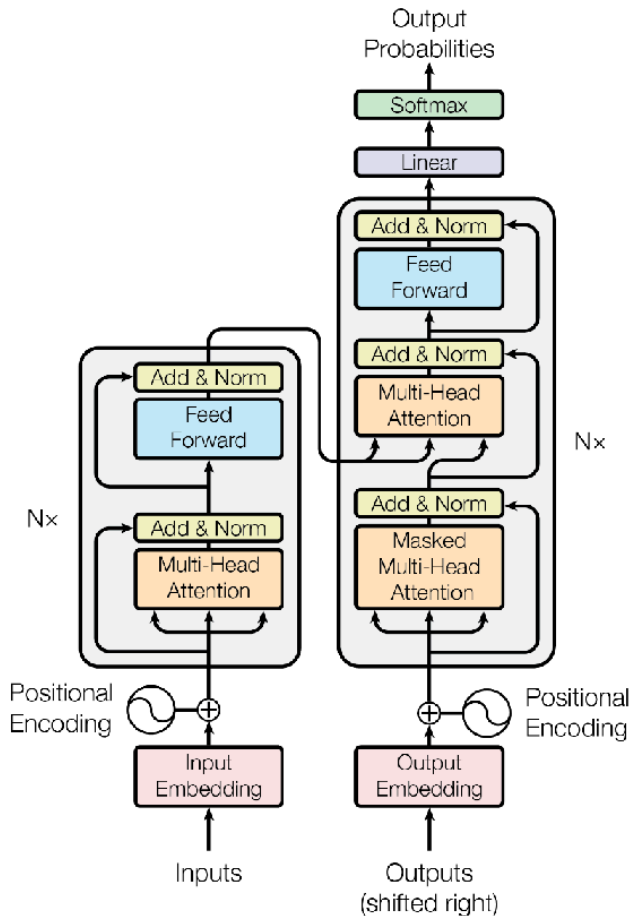
- LayerNorm: **Ba et al., 2016** normalize each vector using its own features.
- Original Transformer: **Vaswani et al., 2017** introduced the standard Transformer block.
- Pre-Norm: **Baevski & Auli, 2018** early use of LayerNorm before attention/MLP.
- Why Pre-Norm: **Xiong et al., 2020** explained its training stability advantages.
- RMSNorm: **Zhang & Sennrich, 2019** simpler norm based on root-mean-square only.

Attention Is All You Need

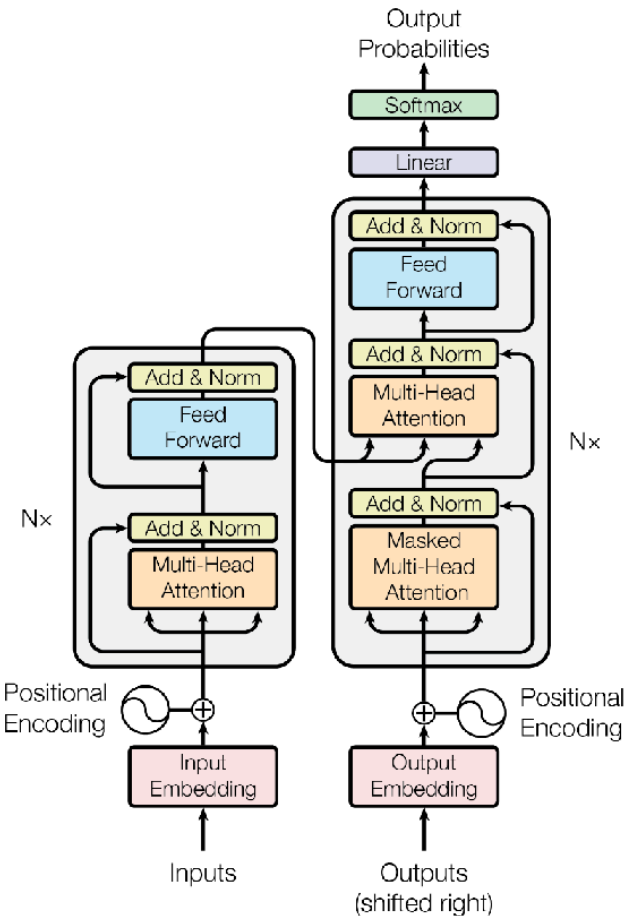


Where Transformers are being used in 2026:

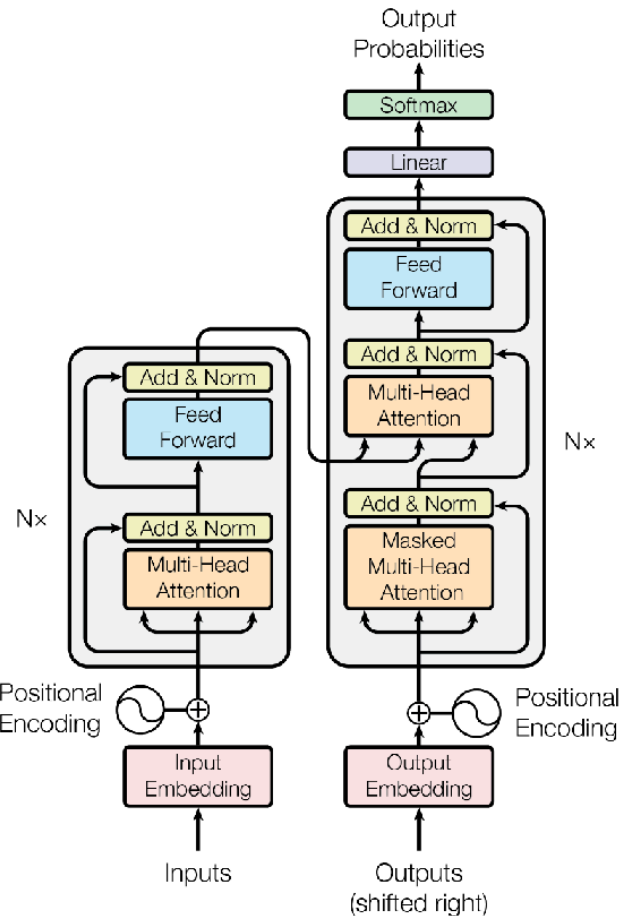
Language models



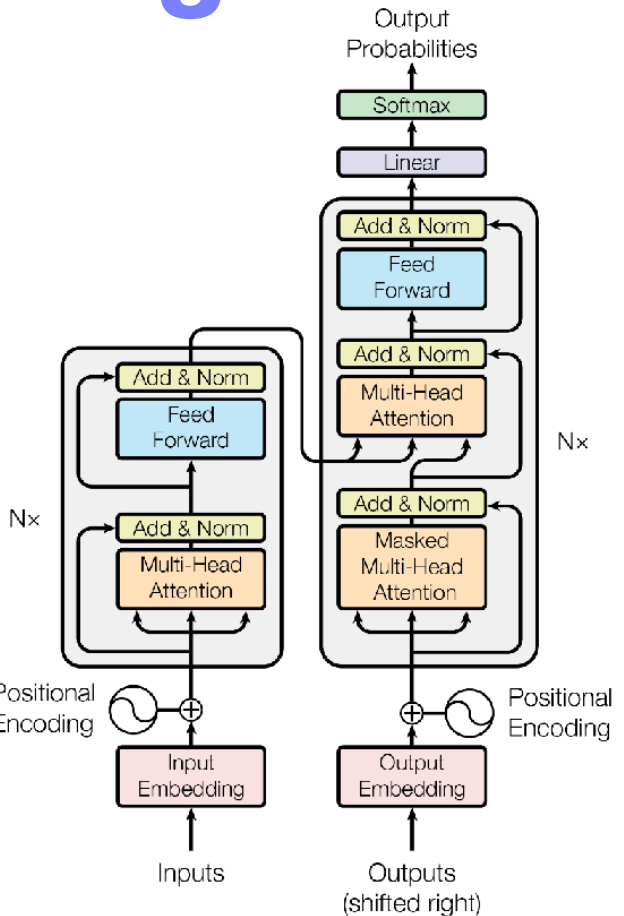
Translation



Speech



Computer vision (On grid data)



Graph-structured data

molecules
interaction networks,
knowledge graphs

...

Genomics sequences

regulatory prediction
genome language models

...

Protein sequences

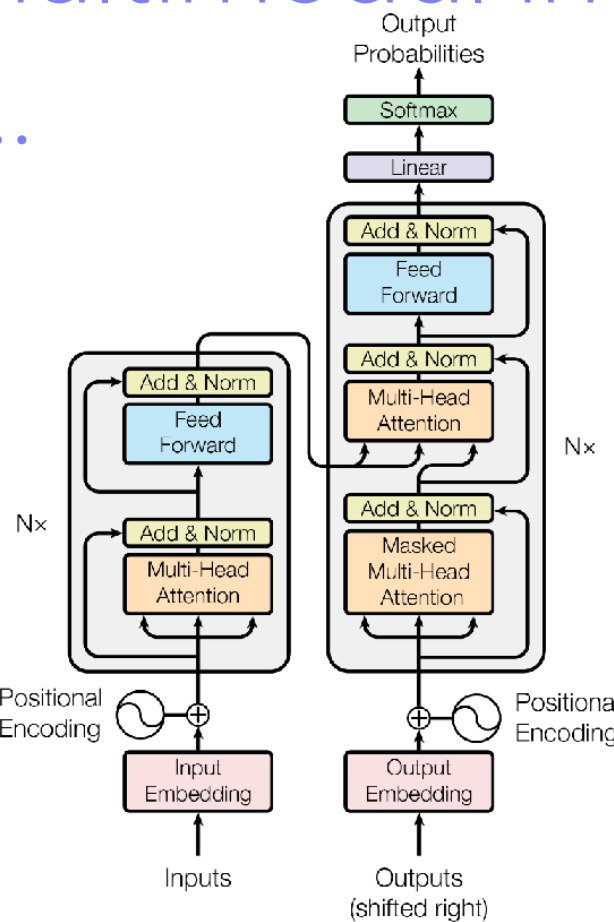
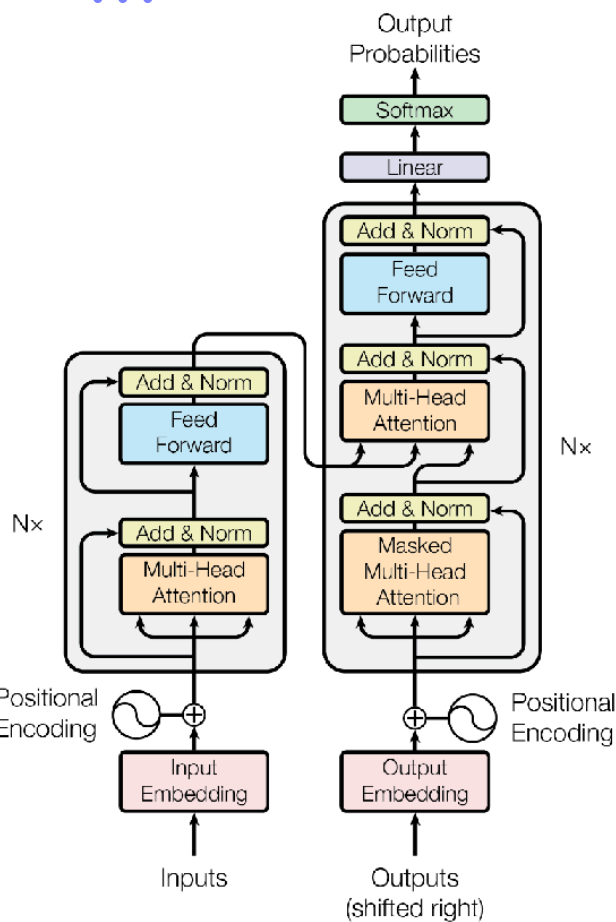
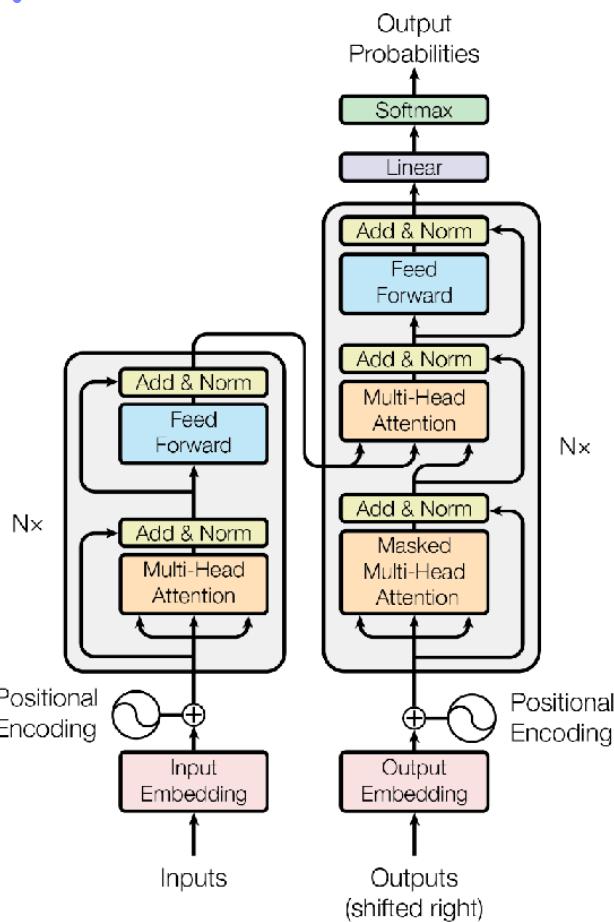
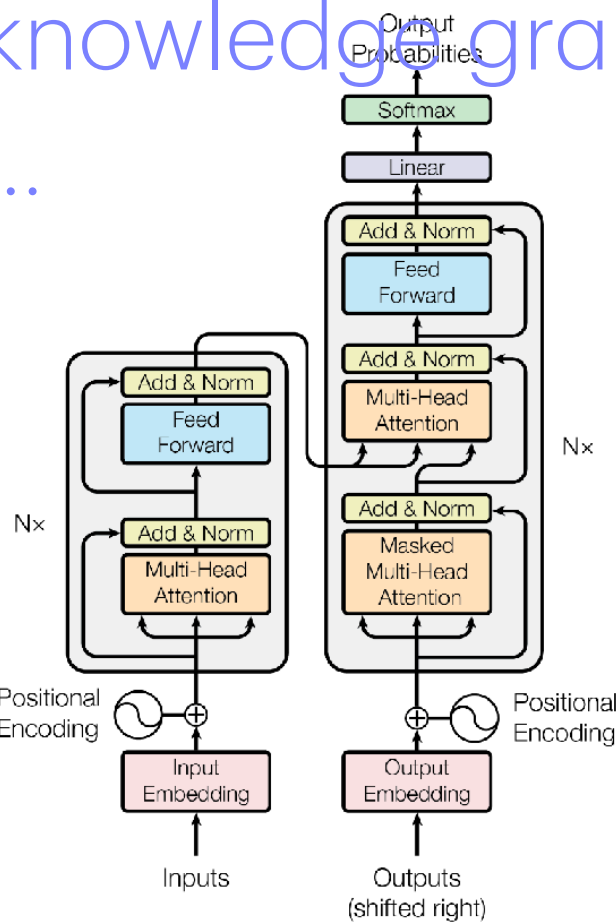
structure prediction
protein design

...

Single cell omics

cell representation
annotation,
multimodal integration

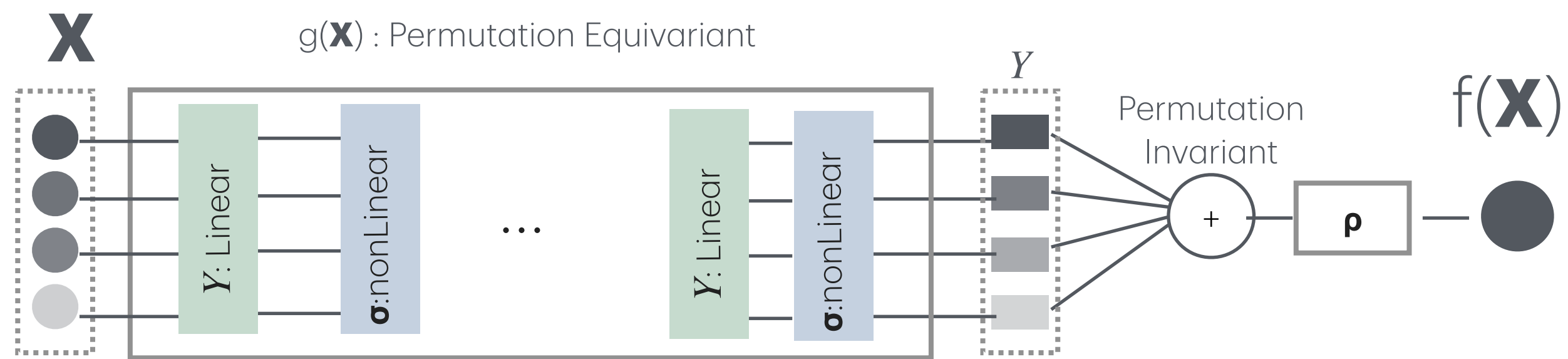
....



1) Transformers naturally fit **sets**

Transformers are naturally fit to process sets

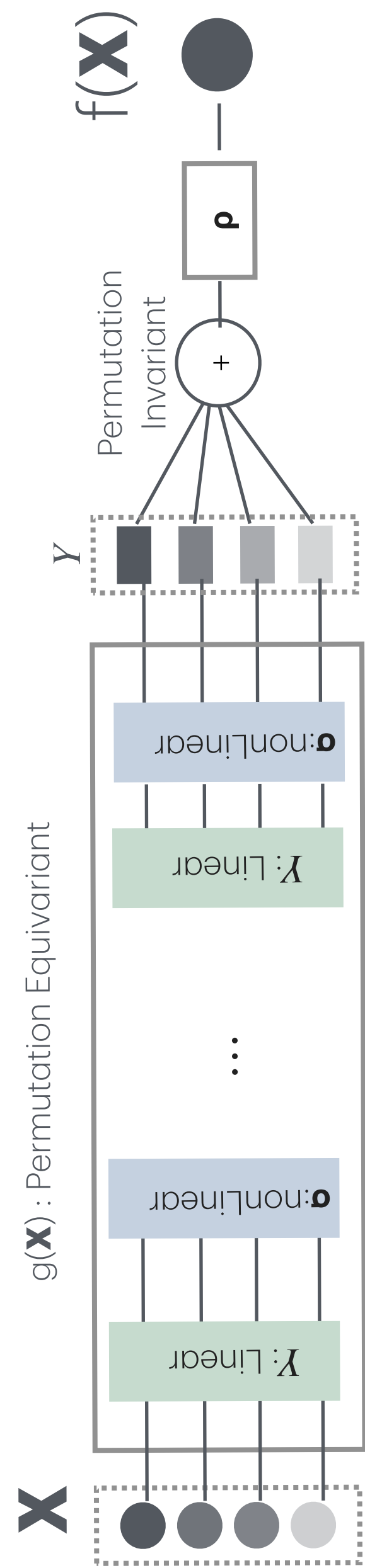
- Why is that? First remember our deep set architecture:



Transformers naturally process set-structured data

- Why is that? First remember our deep set architecture:

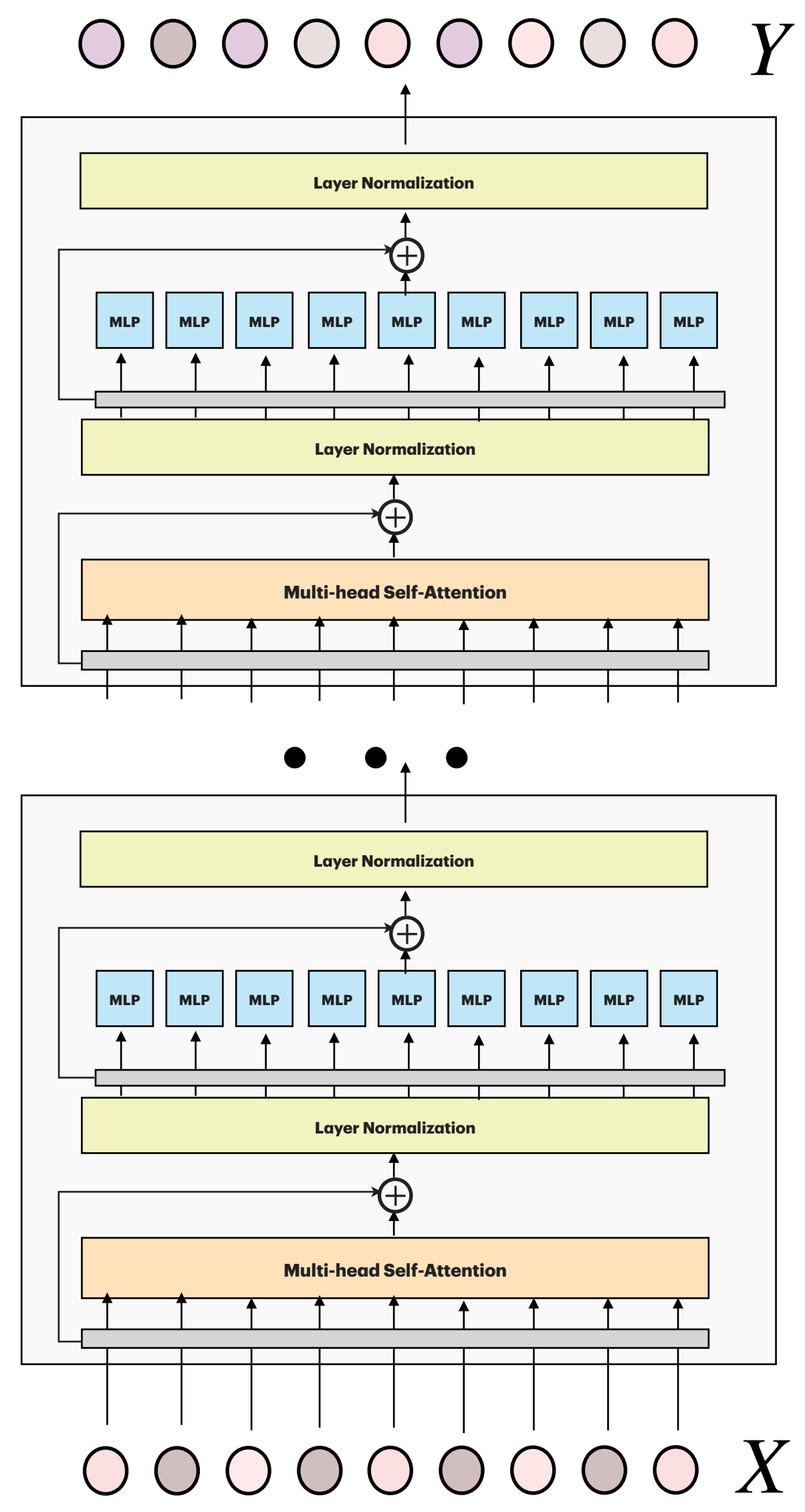
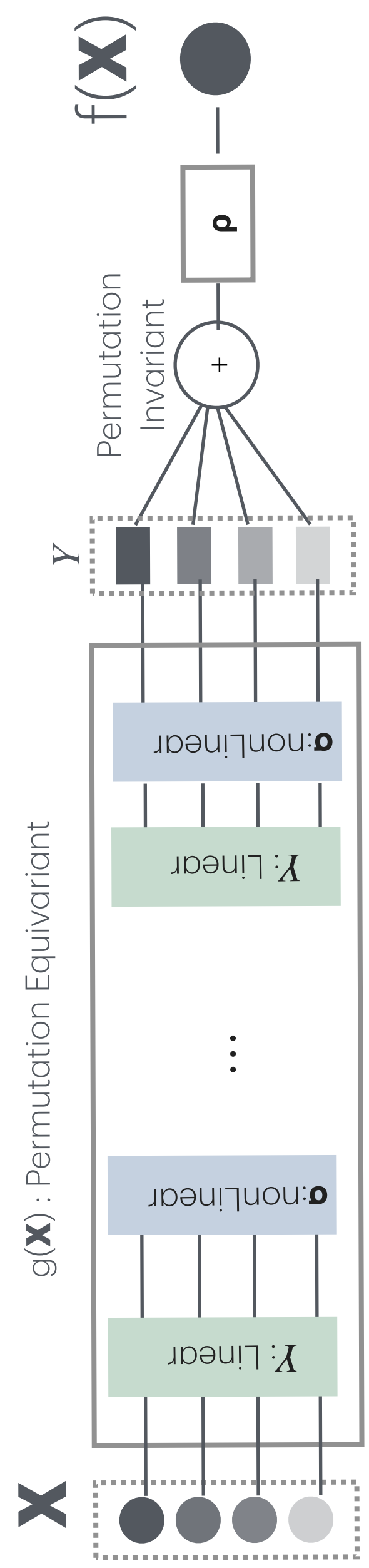
$$f(X) = \rho \left(\sum_{x \in X} g(x) \right)$$



Transformers naturally process set-structured data

- Why is that? First remember our deep set architecture:

$$f(X) = \rho\left(\sum_{x \in X} g(x)\right)$$



Transformers naturally process set-structured data

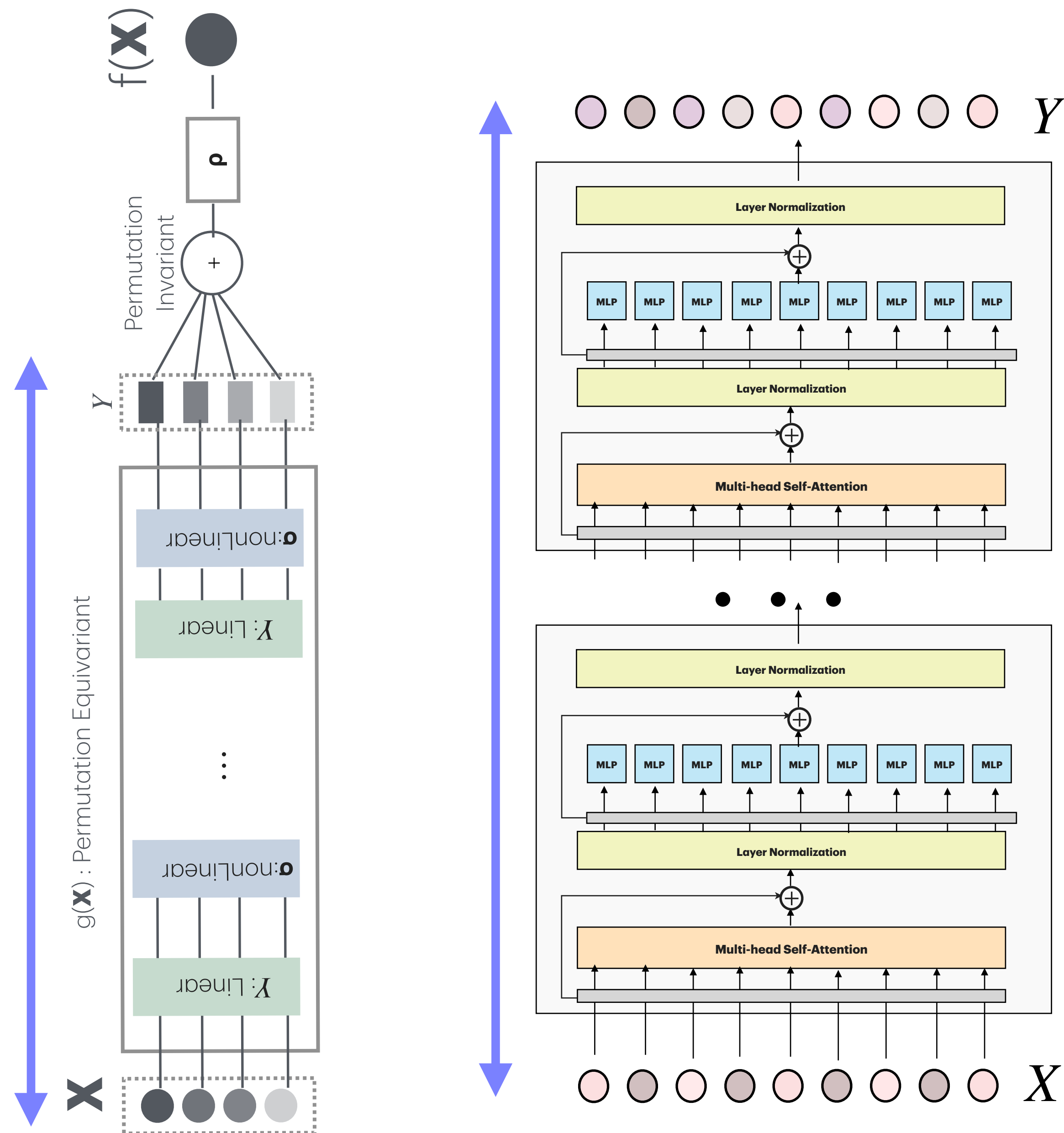
- Why is that? First remember our deep set architecture:

$$f(X) = \rho \left(\sum_{x \in X} g(x) \right)$$

- This means Transformer blocks can be used naturally as $g(x)$ in our deep sets!

Both Permutation Equivariant

- Deep Sets need a permutation-equivariant set-to-set map; Transformer blocks provide exactly that.



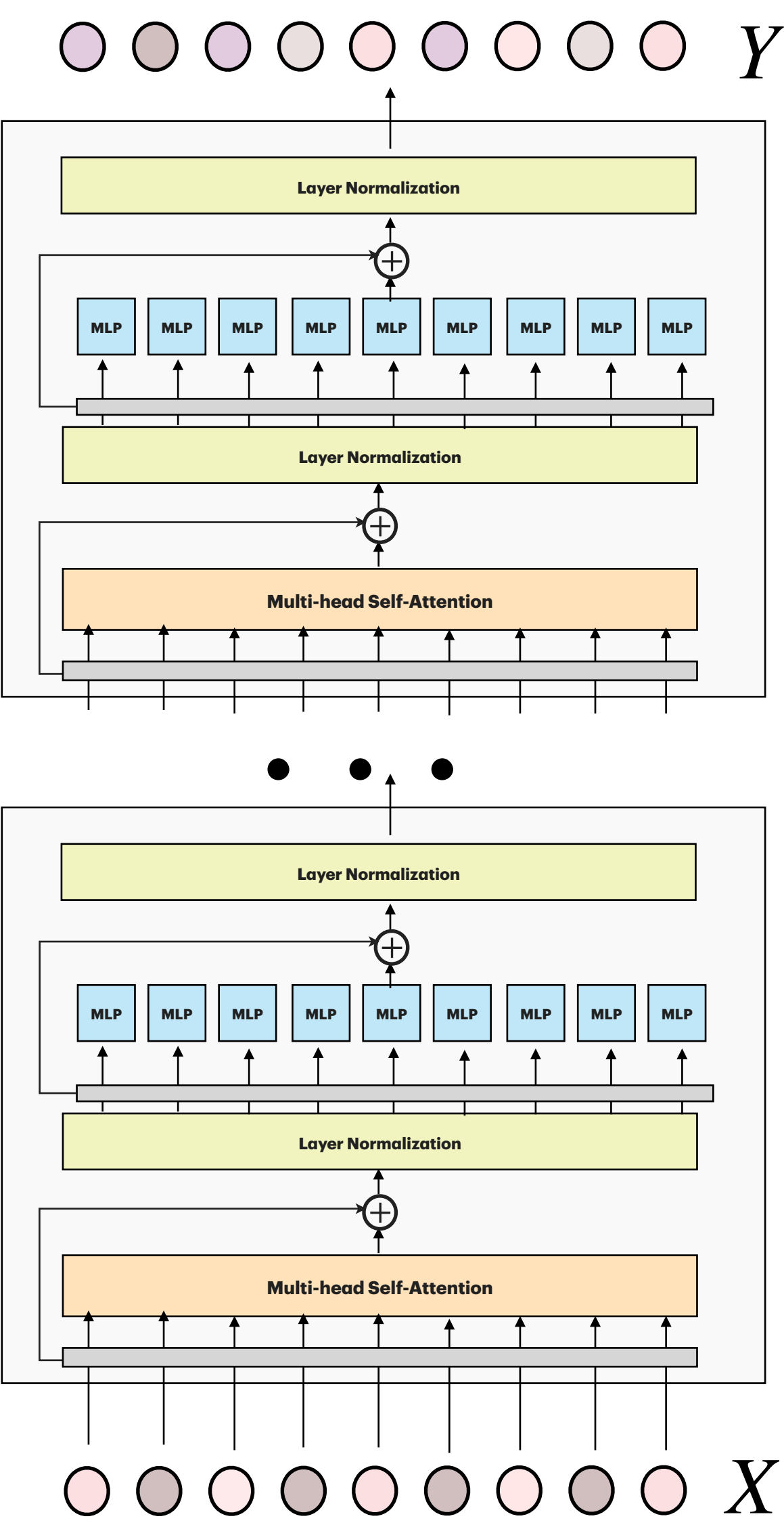
Transformers naturally process set-structured data

- Attention (the core mechanism of Transformers) is naturally a **set operator**:
 - It takes a collection of vector elements in, and returns a collection of vectors out.
 - By itself, it does not know order, grid structure, or graph structure.

Computation phase: Each updated vector element goes through an individual transformation via MLP

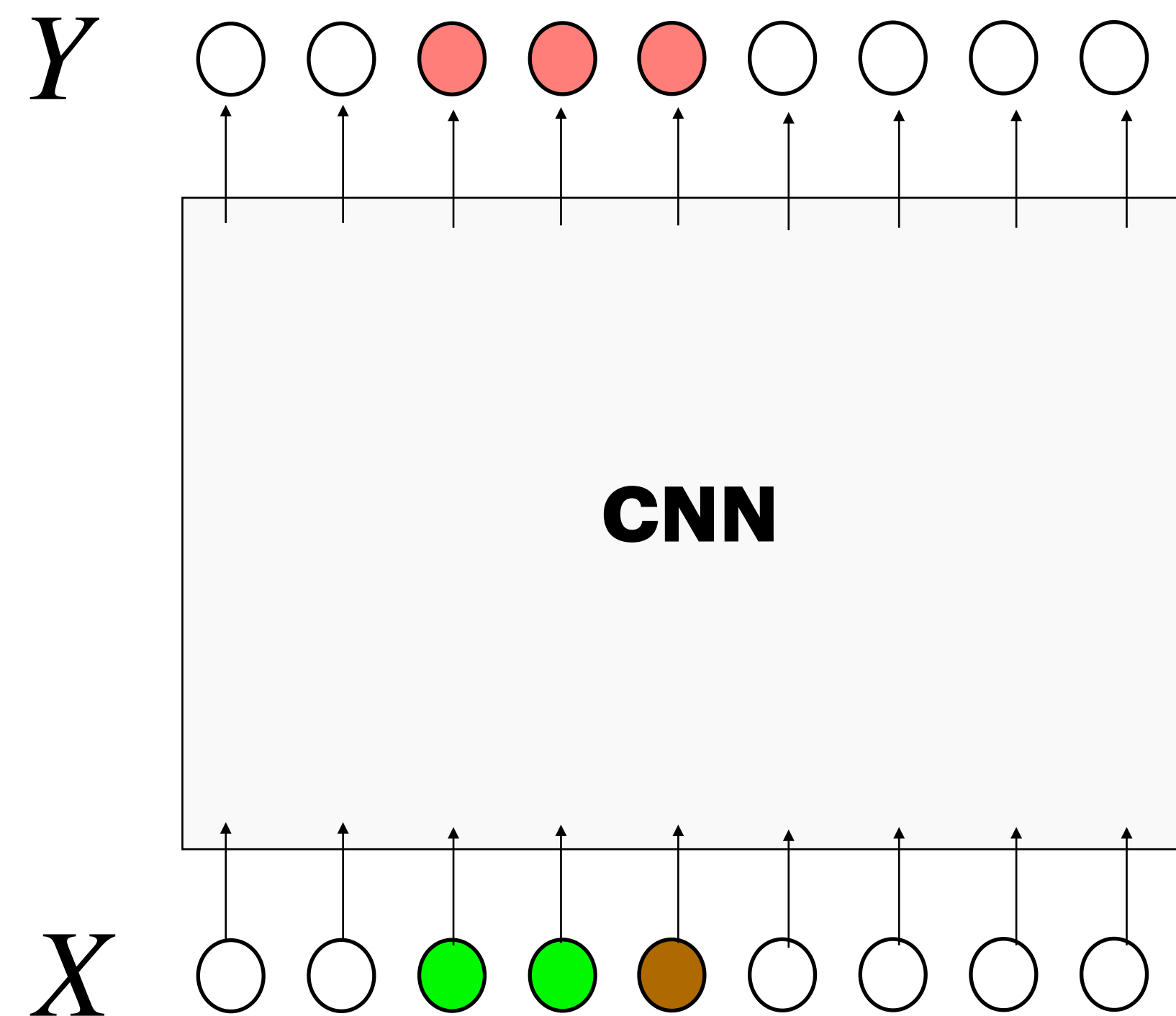
Communication Phase: Each vector element communicates with all elements and update itself

Gets the input set of vector elements

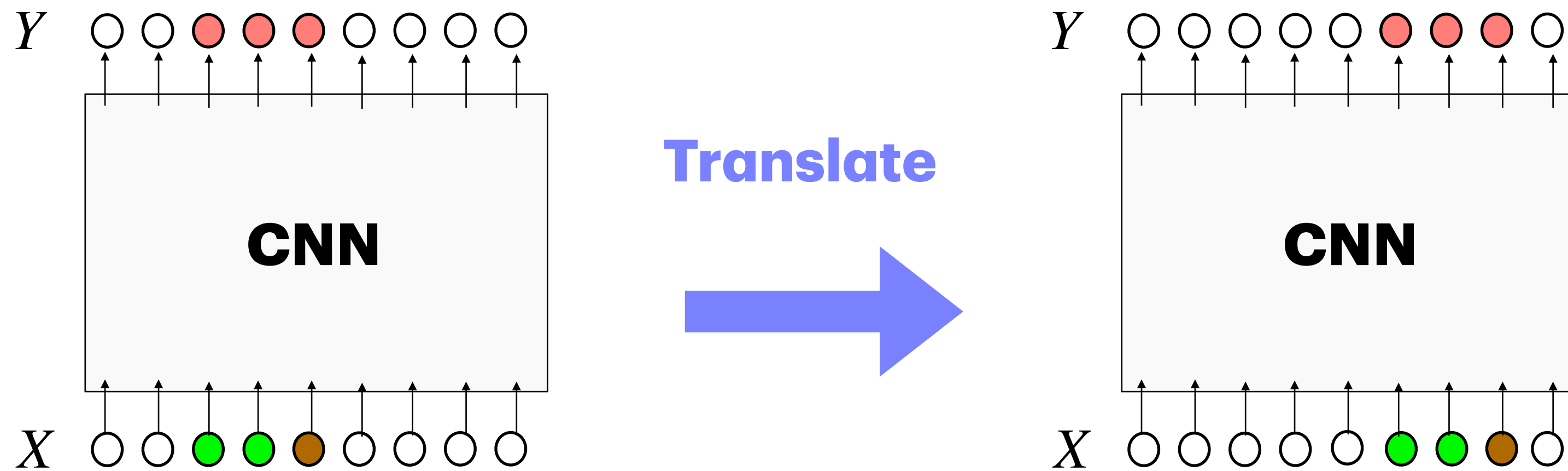


2) For ***sequences*** / ***grids***, we add positional encoding

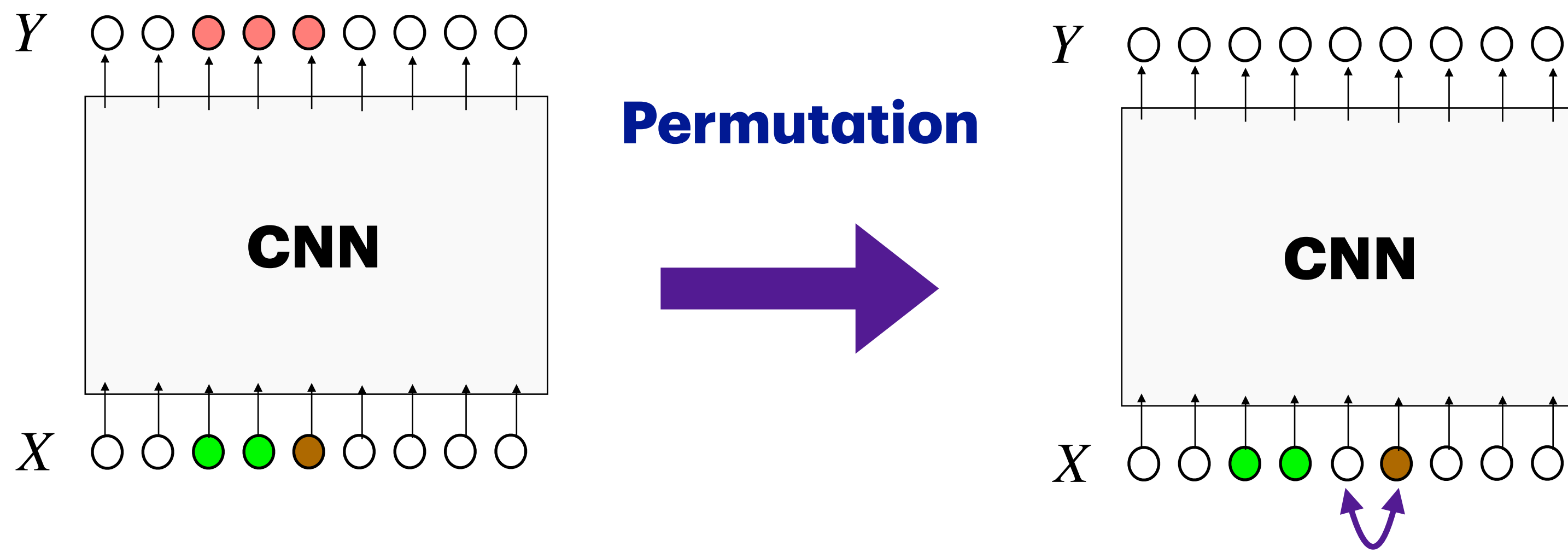
- In this section, our goal is to make some changes and tweak to the vanilla Transformer and make it more suitable to process grids (e.g. images) and sequences (e.g. text, DNA, etc).
- But before doing those tweaks, we want to take few steps back, and have a deeper view of the situation!
- Let's remind ourselves about what was the essence of **CNNs**:
 - Consider a CNN that does segmentation
(for example for segmentation tasks, where we want to find **where a tree is**)



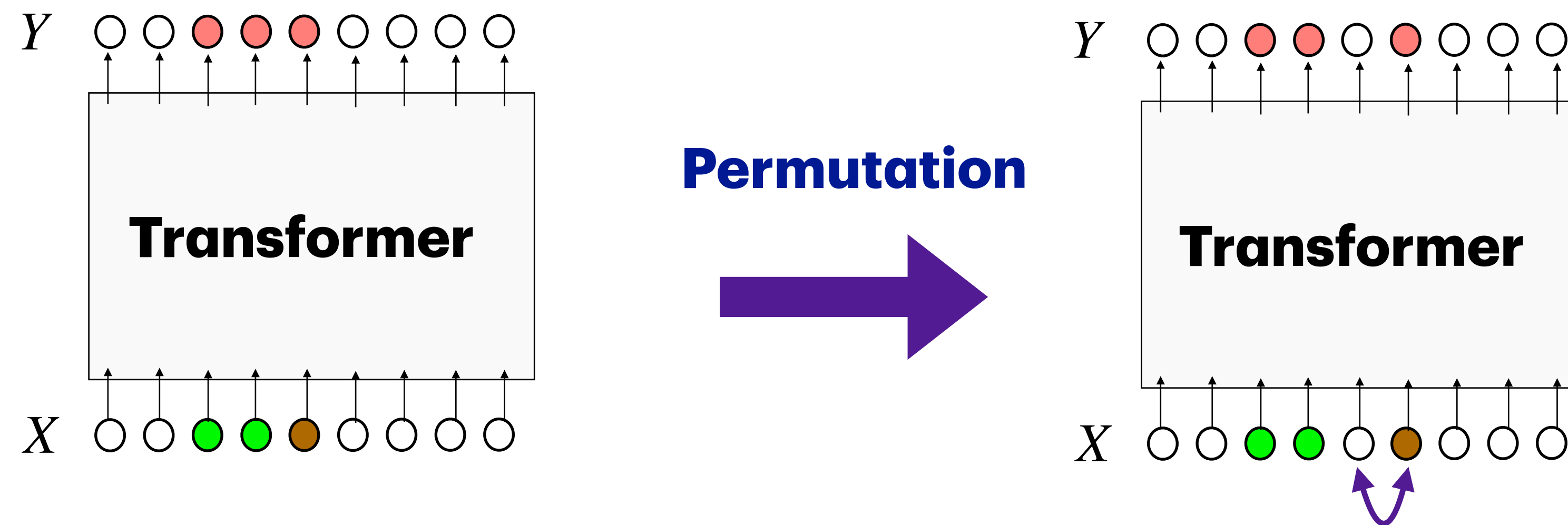
- Consider a CNN that does segmentation
(for example for segmentation tasks, where we want to find **where a tree is**)
- The CNN is **Translation (T) Equivariant**: $CNN(T.X) = T.CNN(X)$



- But, CNN is not **permutation (P) equivariant** in general: $CNN(P.X) \neq P.CNN(X)$



- Now we can ask the same question about **Transformers**.
- Consider a Transformer on a similar input and on a similar segmentation task, where we want to find **where a tree is**.
- The Transformer is **permutation (P) equivariant**: $Transformer(P \cdot X) = P \cdot Transformer(X)$



This is not good! Because there is no three where it predicted to be, but transformer has no way to understand local interactions, it sees the input as just an unstructured set!

"Transformers: Distance-Blind Global Attention"

What about **translation**?

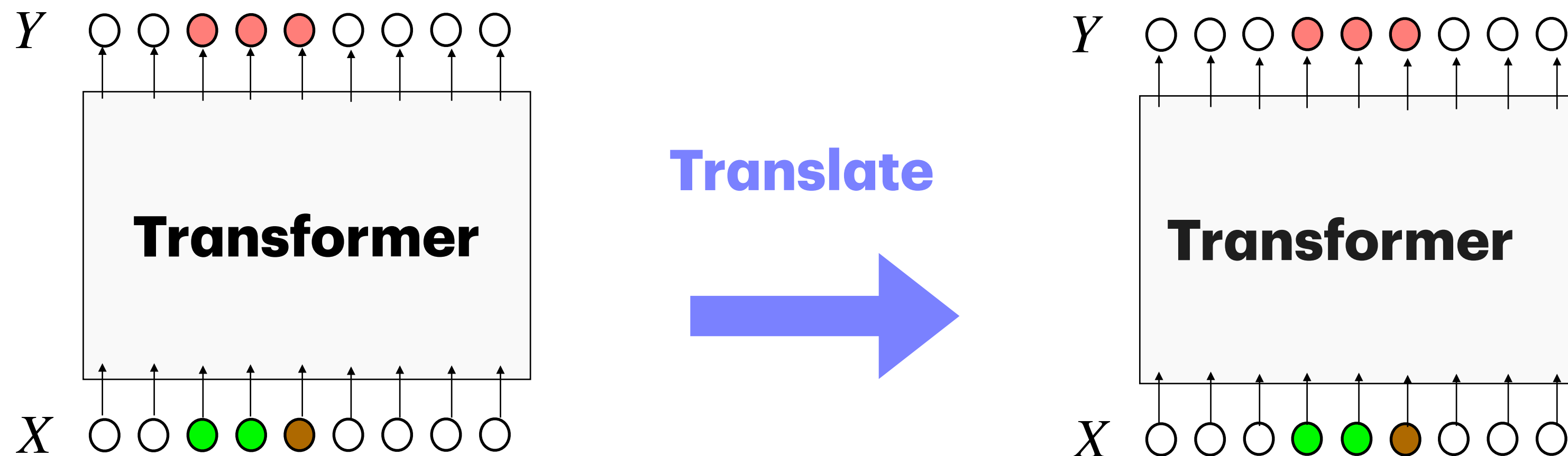
The key to investigate the effect of Translation (T) on transformers is:

“A circular Translation can be represented as a permutation of positions”



- Because (circular) Translation is basically a specific permutation:

- **Translation (T) Equivariant:** $Transformer(T.X) = T.Transformer(X)$



To summarize, for grid data:

We want Translation (T) Equivariance

👍 CNN allows translation equivariance

👍 **Transformer** allows translation equivariance

We don't want permutation (P) equivariance

👍 CNN does not allow Permutation equivariance

👎 **Transformer** allows Permutation equivariance

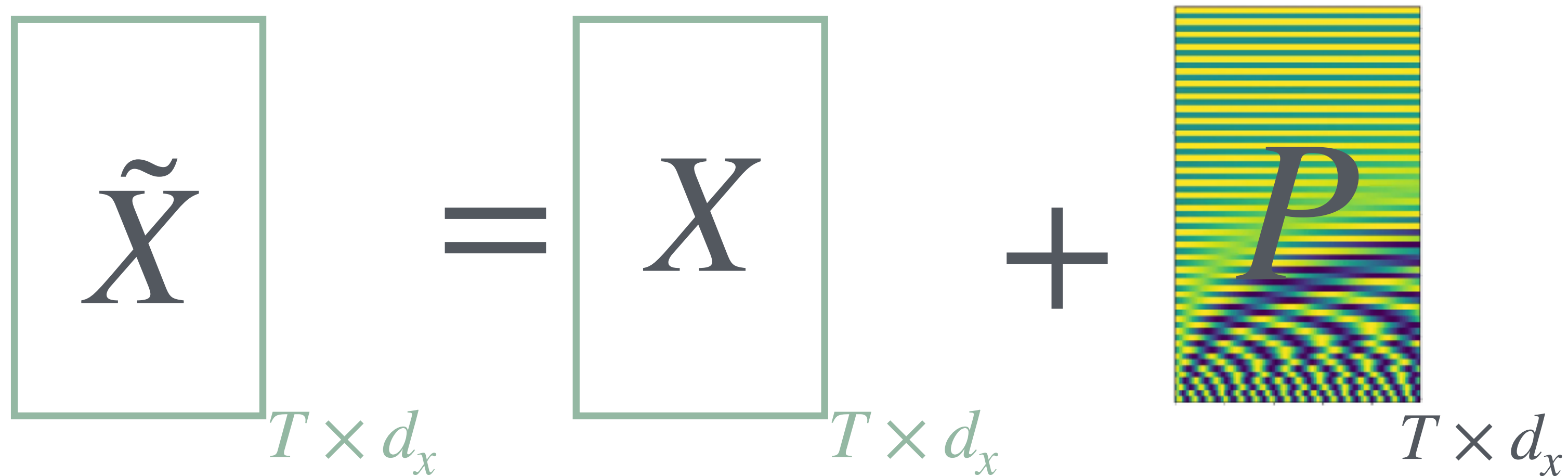
- Plain self-attention is permutation equivariant, that means it is naturally a set operator
- For sequences and grids, this symmetry is **too strong**.
- Transformer model is **geometry-blind**: it does not know before/after near/far row/column and local neighborhoods. It just assumes a bag of vectors.
- More data cannot fix this, because exact architectural symmetry cannot be broken by learning alone

Positional Encoding (PE)

- How can we solve this to be able to use Transformers in grids and sequences?
 - A nice trick: Positional encoding:
 - We inject location into each element by adding a positional vector:

$$\tilde{x}_i = x_i + p_i$$

- Now two identical elements at different positions are no longer identical to the model

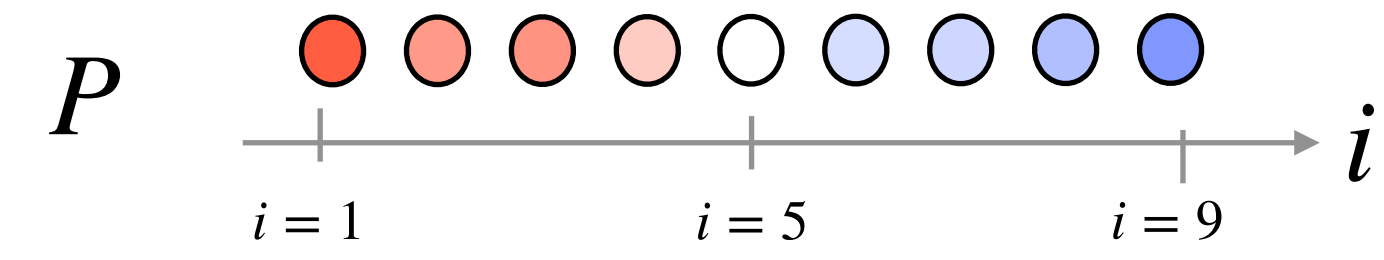
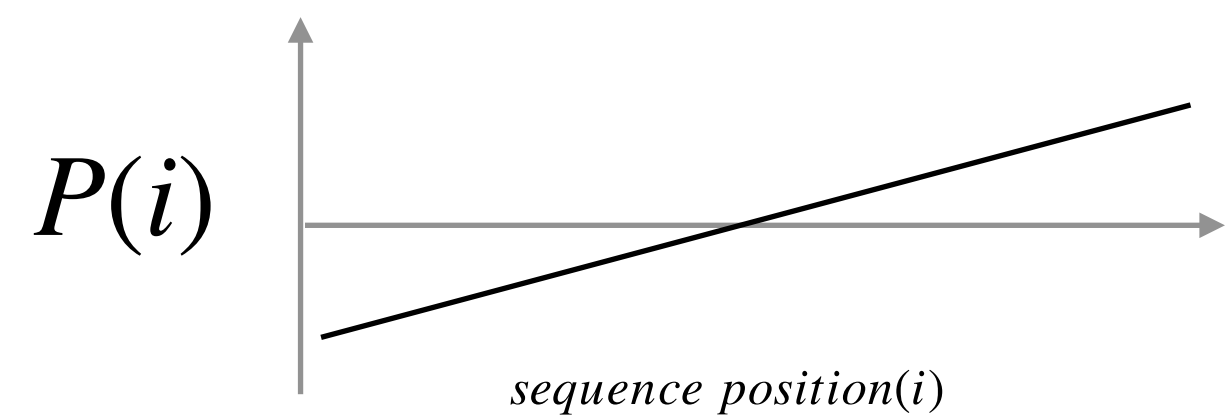


The diagram shows the equation $\tilde{X} = X + P$ where $\tilde{X}$, X , and P are each enclosed in a box. Below each box is the dimension $T \times d_x$. The box for $\tilde{X}$ is on the left, followed by an equals sign, then the box for X , followed by a plus sign, and finally the box for P . The box for P contains a heatmap visualization of a sinusoidal wave, representing the positional encoding vectors.

- **This is the most common way of adding positional encoding to Transformers**

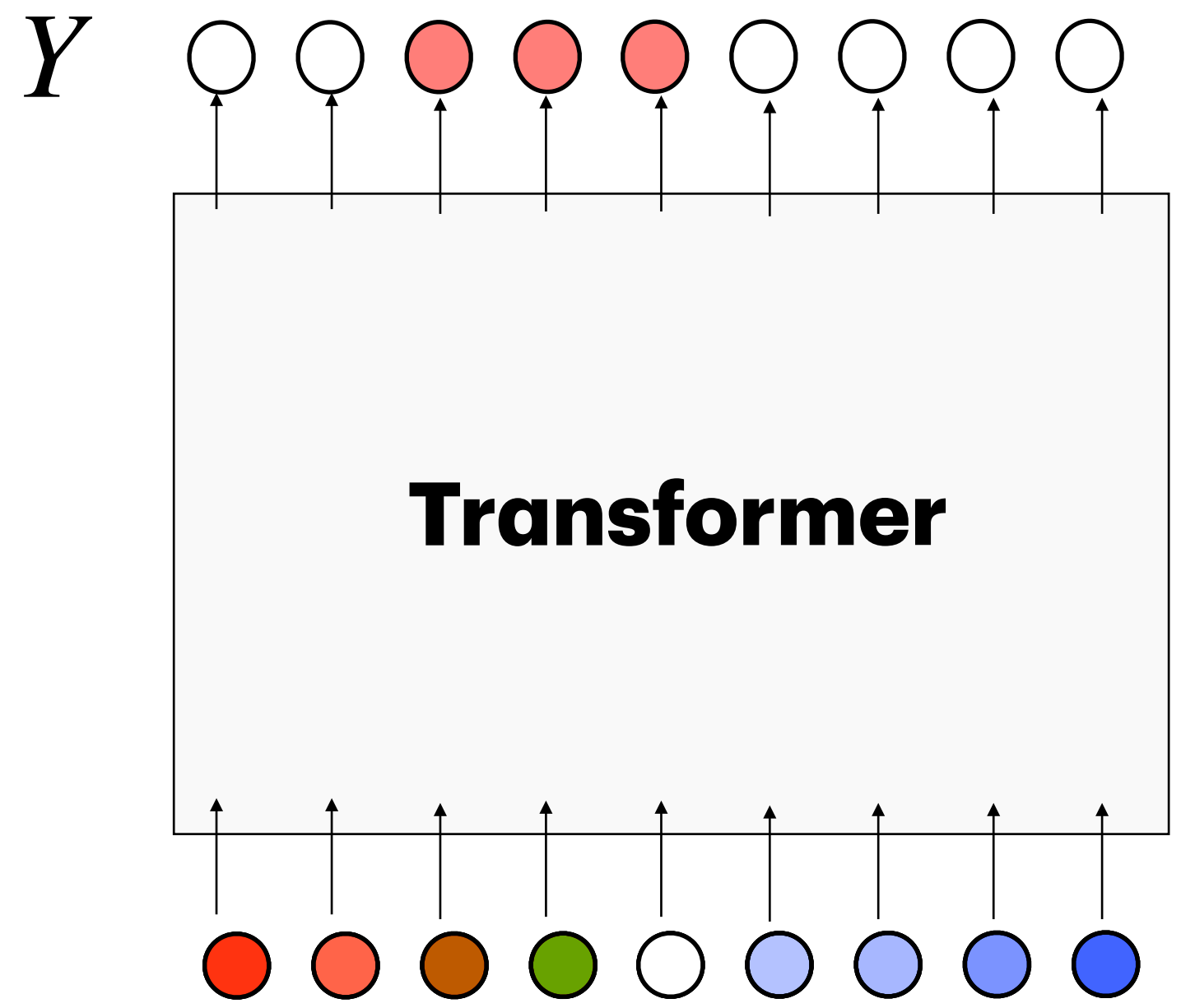
Positional Encoding (PE)

- An illustrative Example:

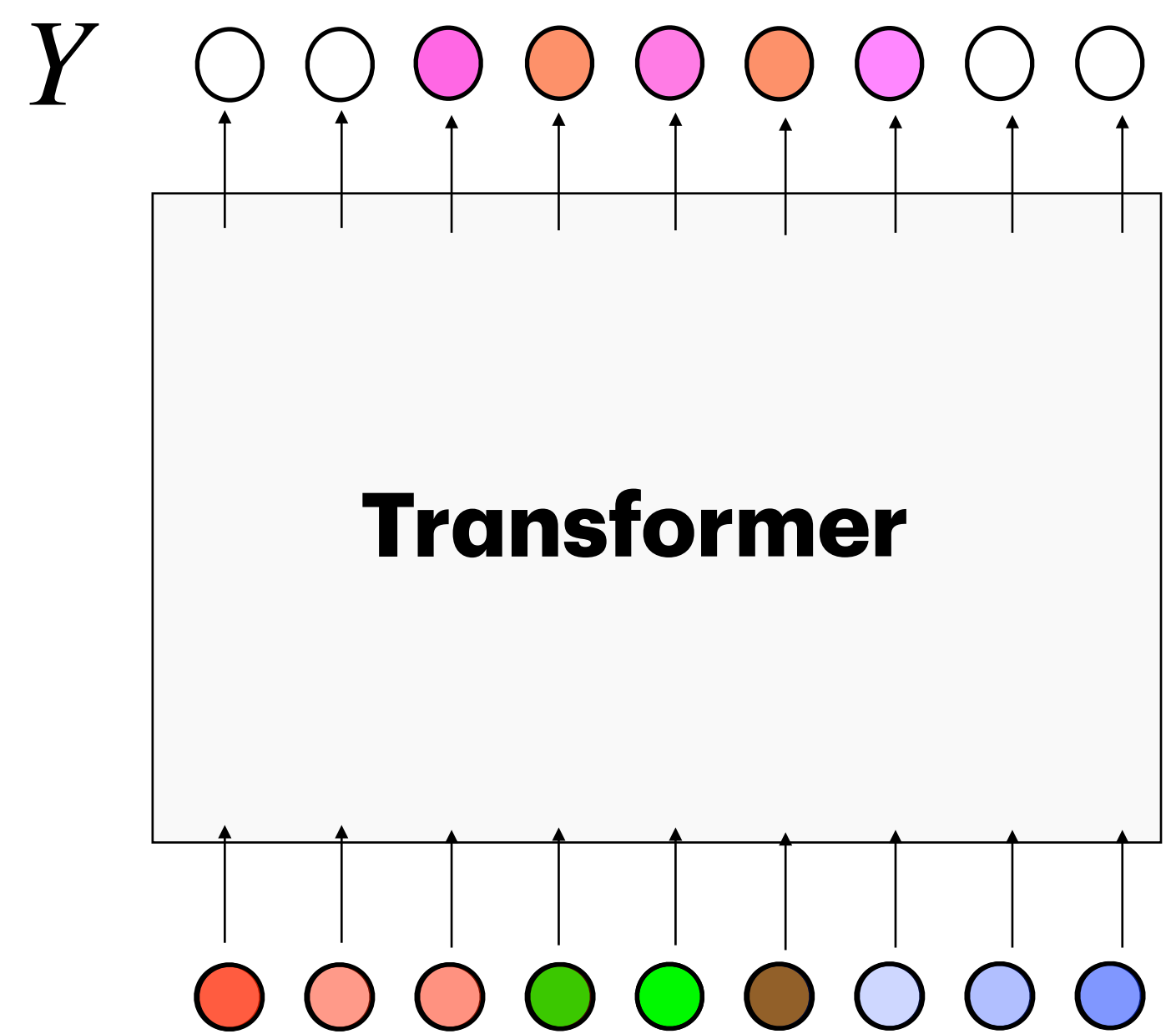


$$\tilde{X} \text{ (red, orange, green, green, white, light blue, light blue, light blue, blue)} = X \text{ (white, white, green, green, brown, white, white, white, white)} + P \text{ (red, orange, orange, orange, white, light blue, light blue, light blue, blue)}$$

- But note that this type of positional encoding **breaks** both **permutation** and **translational** symmetry!
- It means it will takes more data for Transformers to learn relationships.
- But it works because Transformers are very expressive



Translate
→



To summarize, so far we saw:

- Plain self-attention is permutation equivariant so it is geometry-blind.
- For sequences/grids, that symmetry is too strong.
- Positional encoding (PE) fixes this by injecting location.
- But PE breaks translation equivariance as well, so we need more data to train the model

You might ask is there a more efficient way to inject PE? Is there a way to inject positional encoding without braking translation equivariance?

- Yes! **Relative Positional Encoding** (RPE) - used in models like minGPT, T5, and modern efficient transformers.

Relative Positional Encoding (RPE)

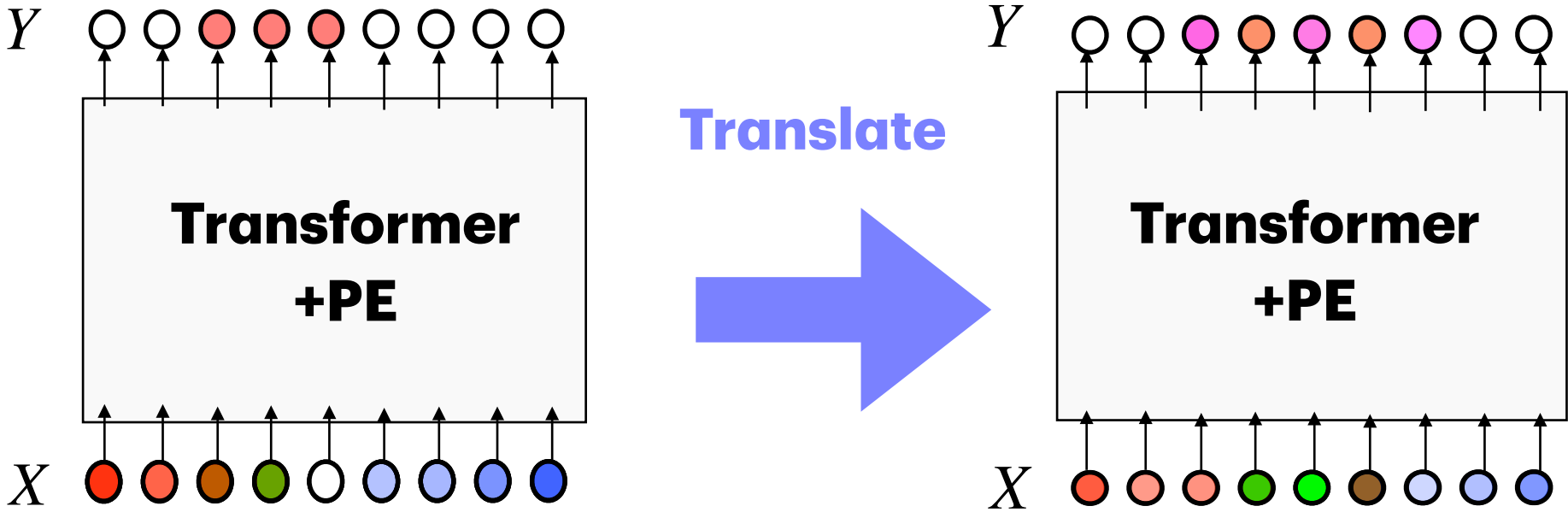
In **PE**: $\tilde{X} = X + P(i) \leftarrow$ Adds position to embeddings
Inside the transformer the Attention layer states intact:

$$Q = \tilde{X}W_Q, \quad K = \tilde{X}W_K, \quad V = \tilde{X}W_V$$

$$Y = \text{Attention}(Q, K, V) = \text{softmax}_{\text{rowwise}} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

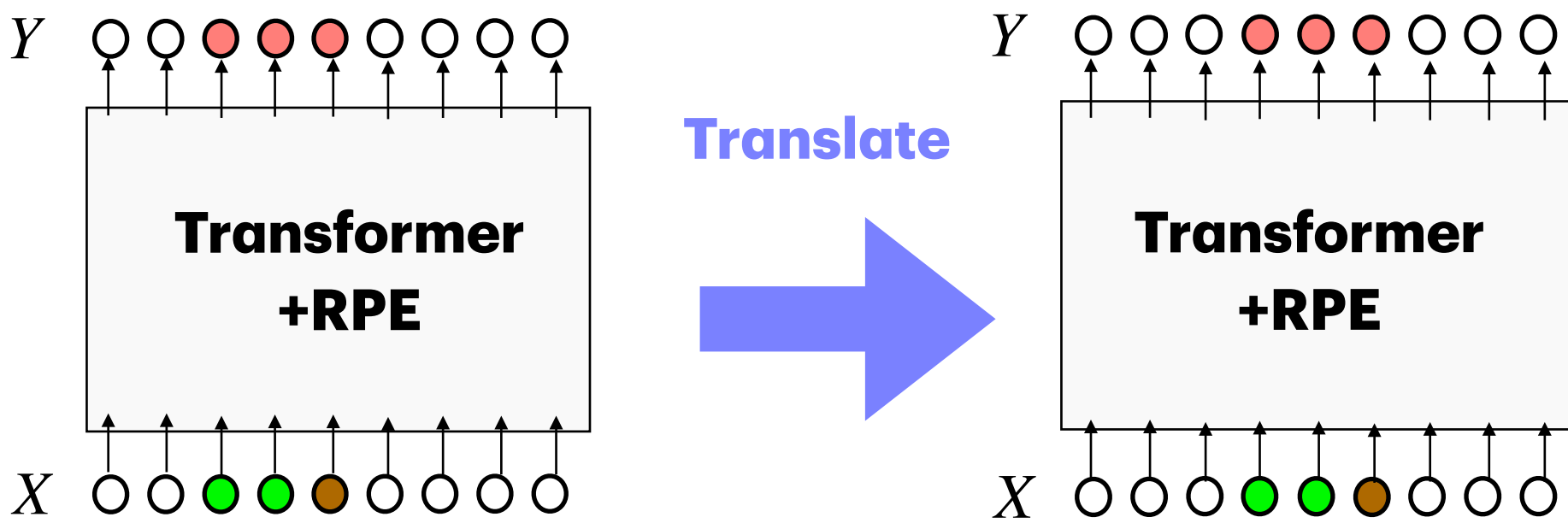
RE based models:

- Break translation equivariance (unlike CNNs)
- Less data-efficient for grids/sequences (must learn same pattern at each position)
- More expressive (can learn position-specific features)
- Simpler to implement and computationally cheaper



In **RPE**: $\tilde{X} = X$ (unchanged), but we modify attention layer.
One possibility is to use this:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$
$$Y = \text{softmax}_{\text{row}} \left(\frac{QK^T}{\sqrt{d_k}} + B^{\text{rel}} \right) V, \quad B_{ij}^{\text{rel}} = b_{i-j}$$



PRE based models are

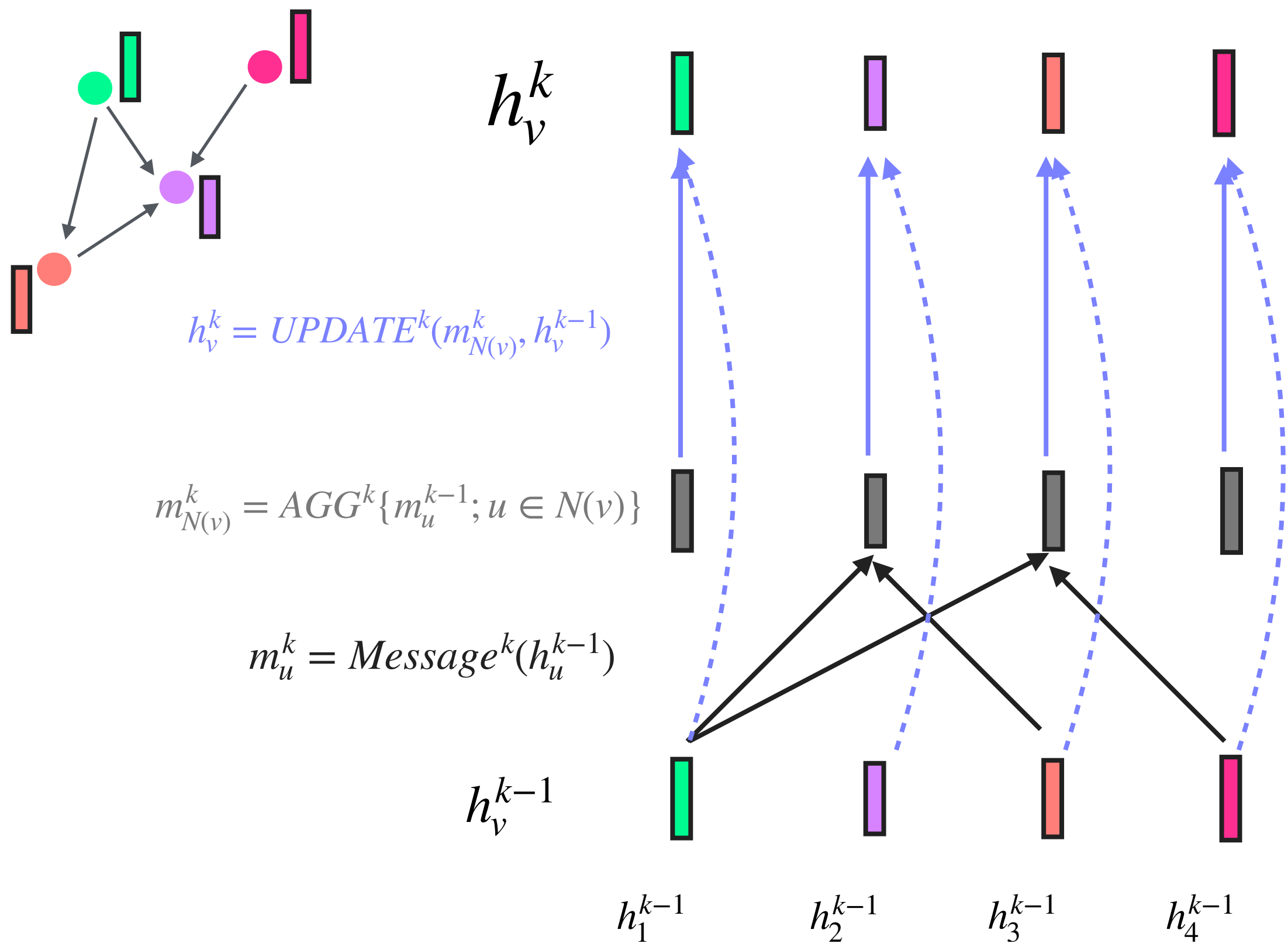
- Translation equivariant (like CNNs)
- More data-efficient for grids/sequences
- Less expressive than PE (can't learn position-specific features)
- Still expressive enough for complex tasks

Shaw, Peter, Jakob Uszkoreit, and Ashish Vaswani. "Self-attention with relative position representations." *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*. 2018.

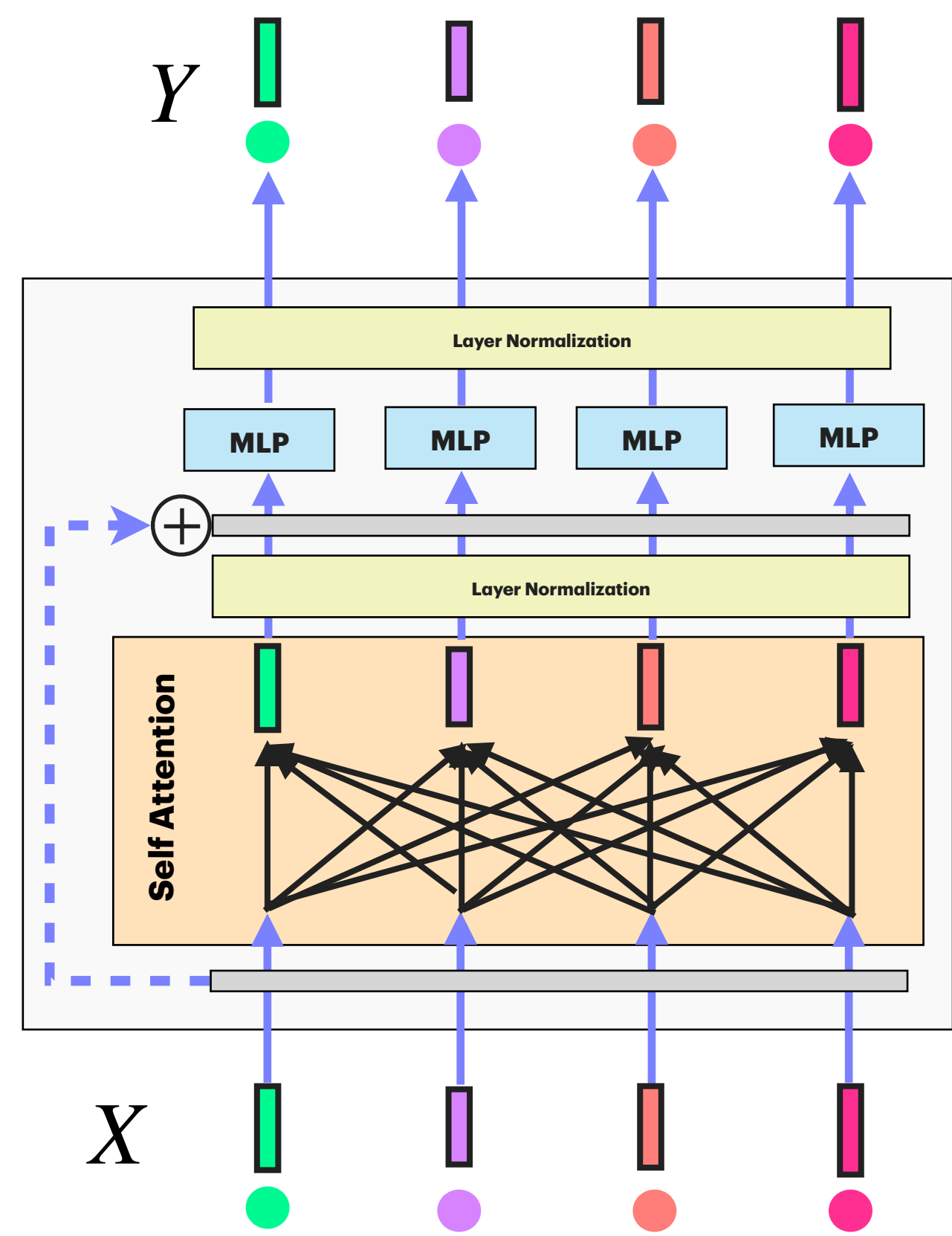
3) For ***sequences and graphs*** we inject graph structure

- Lets remind ourselves about computation graphs of GNNs and Transformers.
- Just looking at it, will be very revealing about what we should focus on to be able to process graphs via Transformers

GNN computation graph

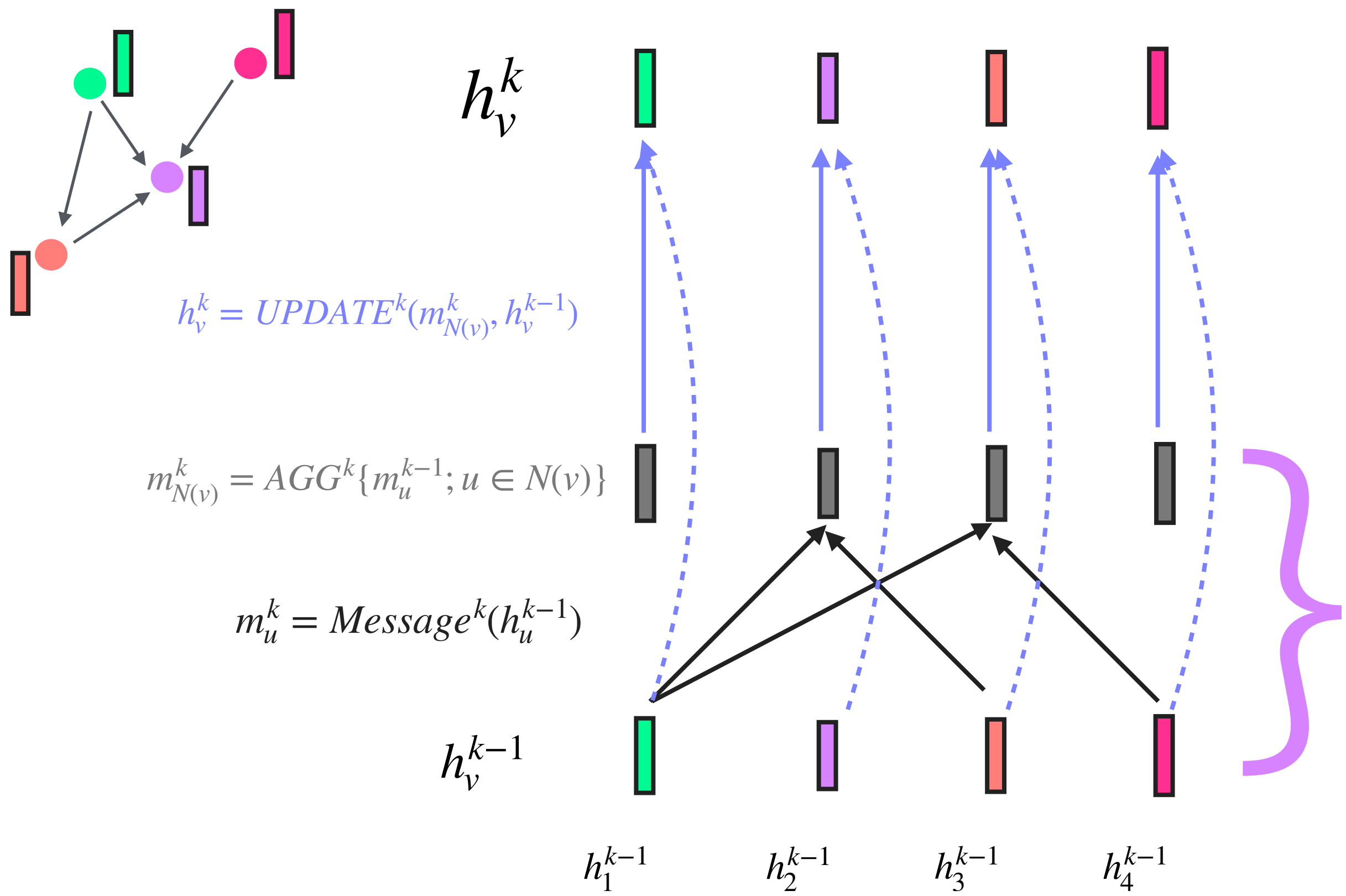


Transformer computation graph



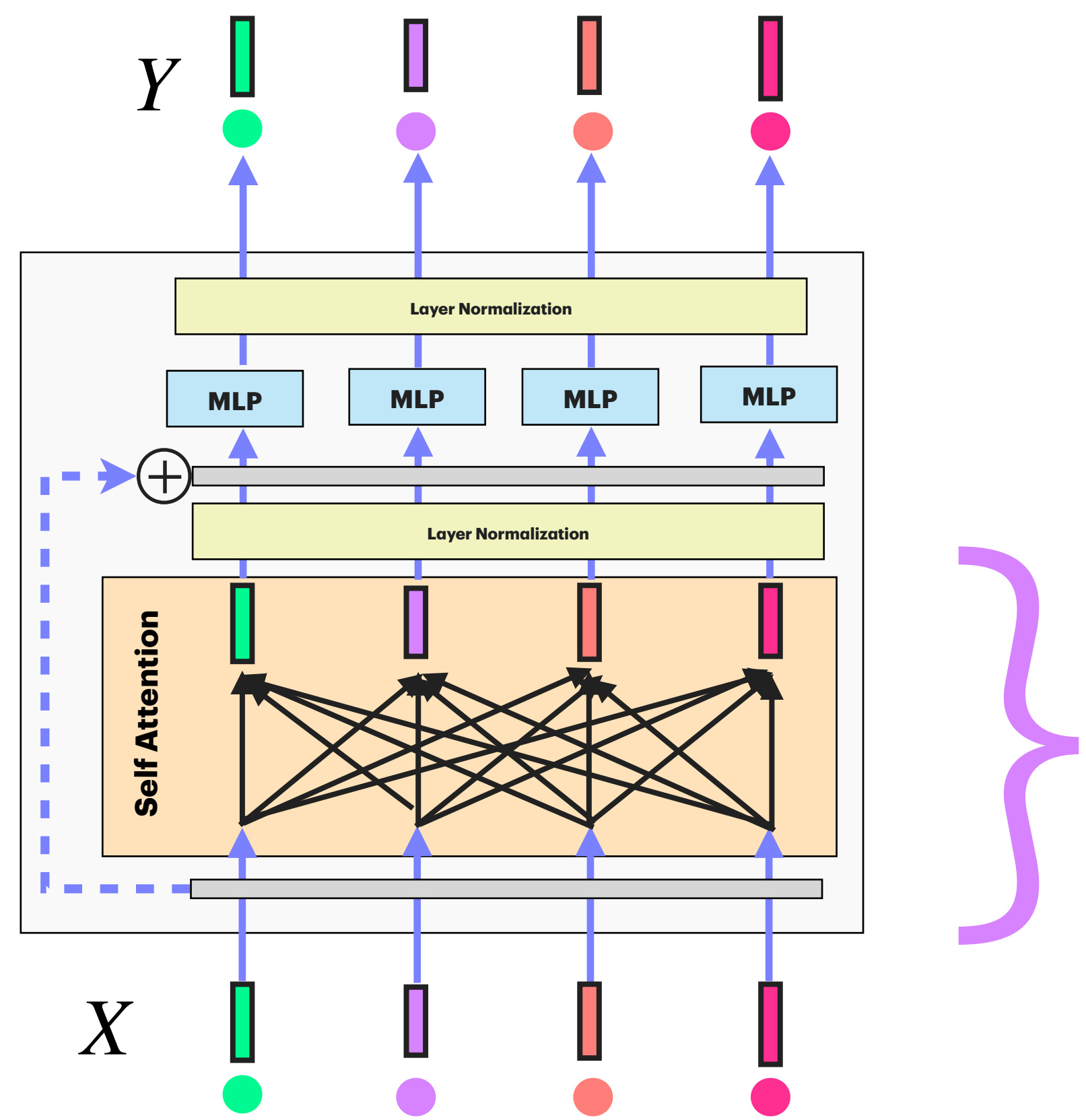
- Lets remind ourselves about computation graphs of GNNs and Transformers.
- Just looking at it, will be very revealing about what we should focus on to be able to process graphs via Transformers

GNN computation graph



Message passing follows graph edges only

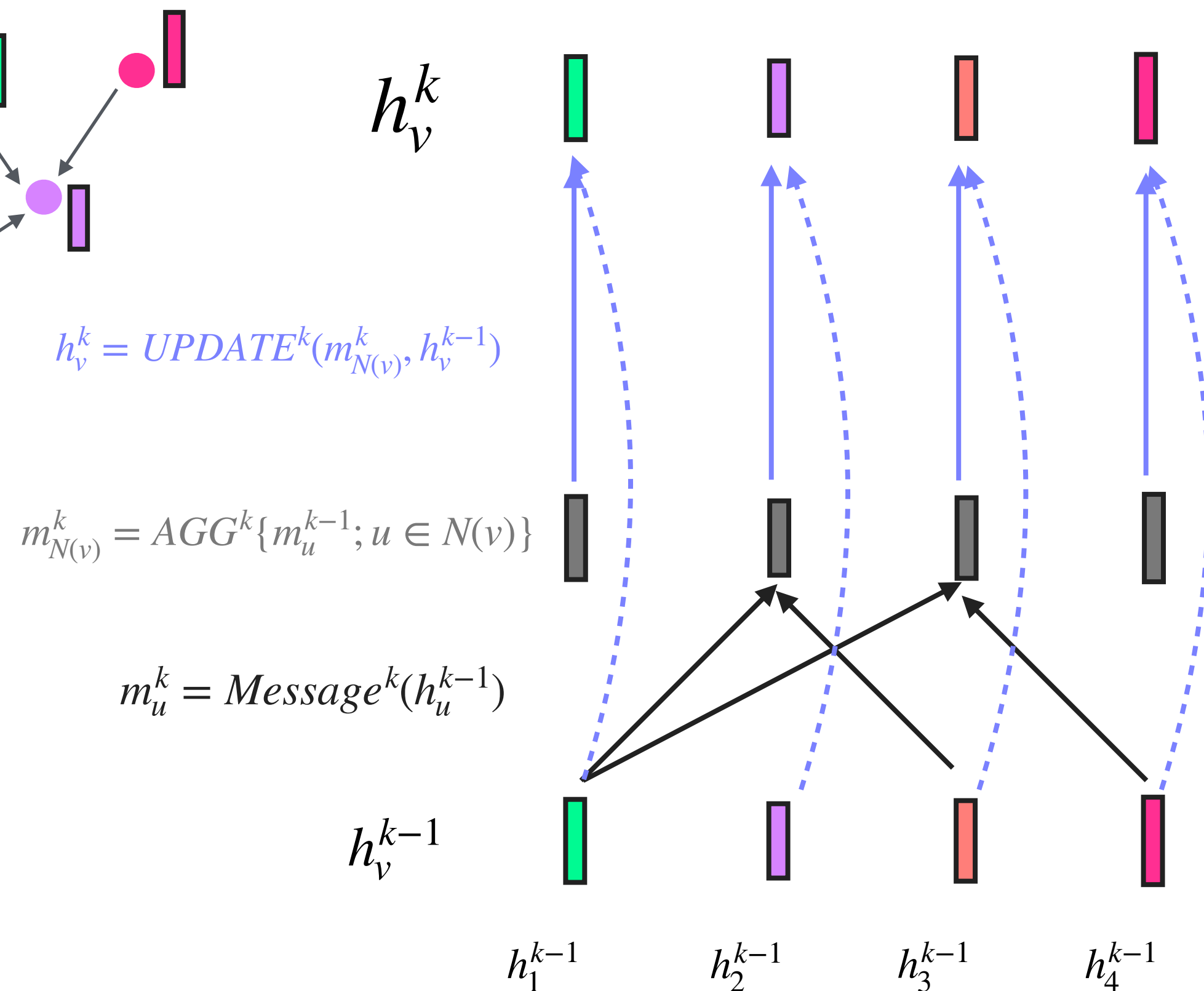
Transformer computation graph



Self-attention is fully connected

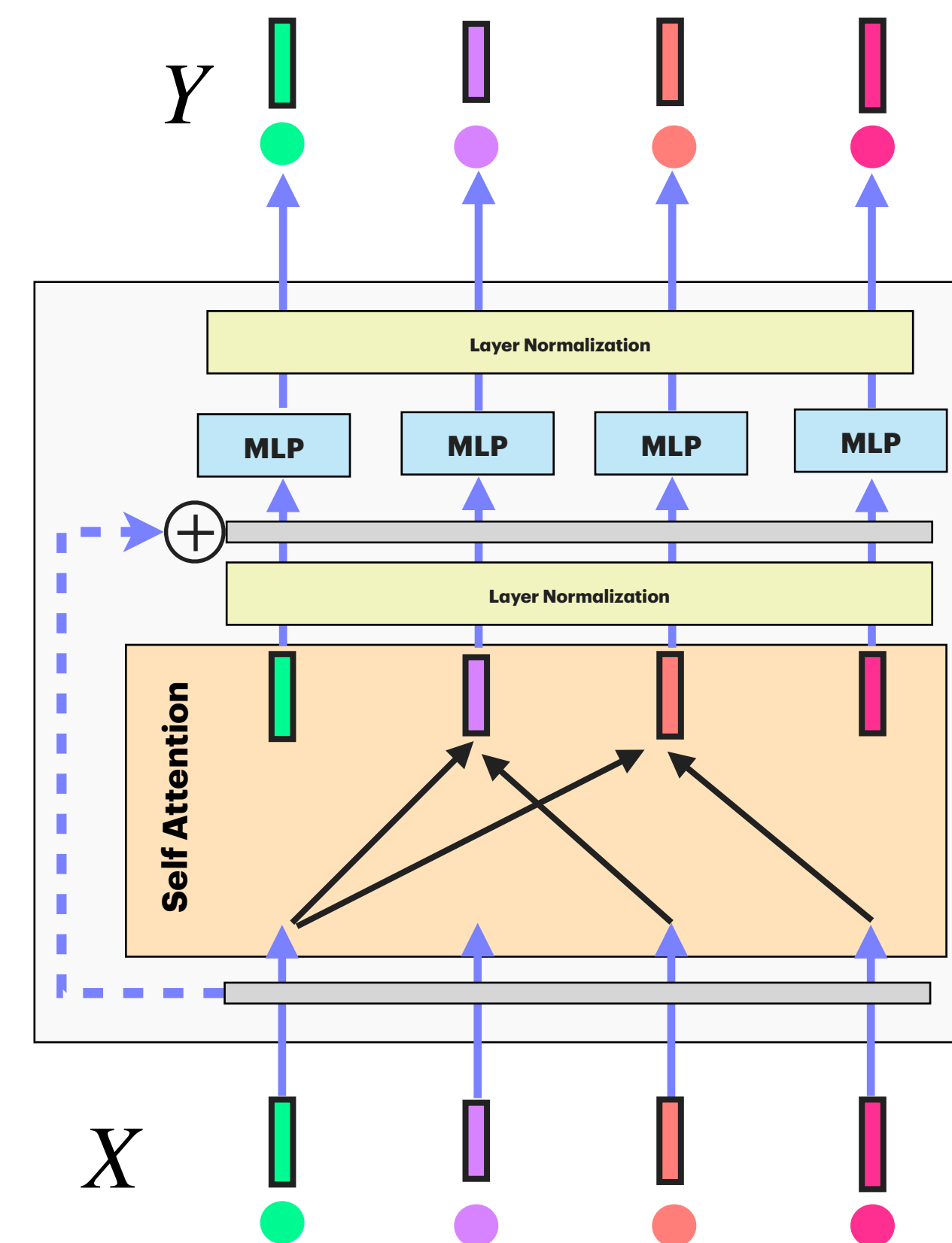
- **The Solution:** Making Transformers Self Attention to only attend to the relevant elements.
- This is called **Masked Attention**

GNN computation graph



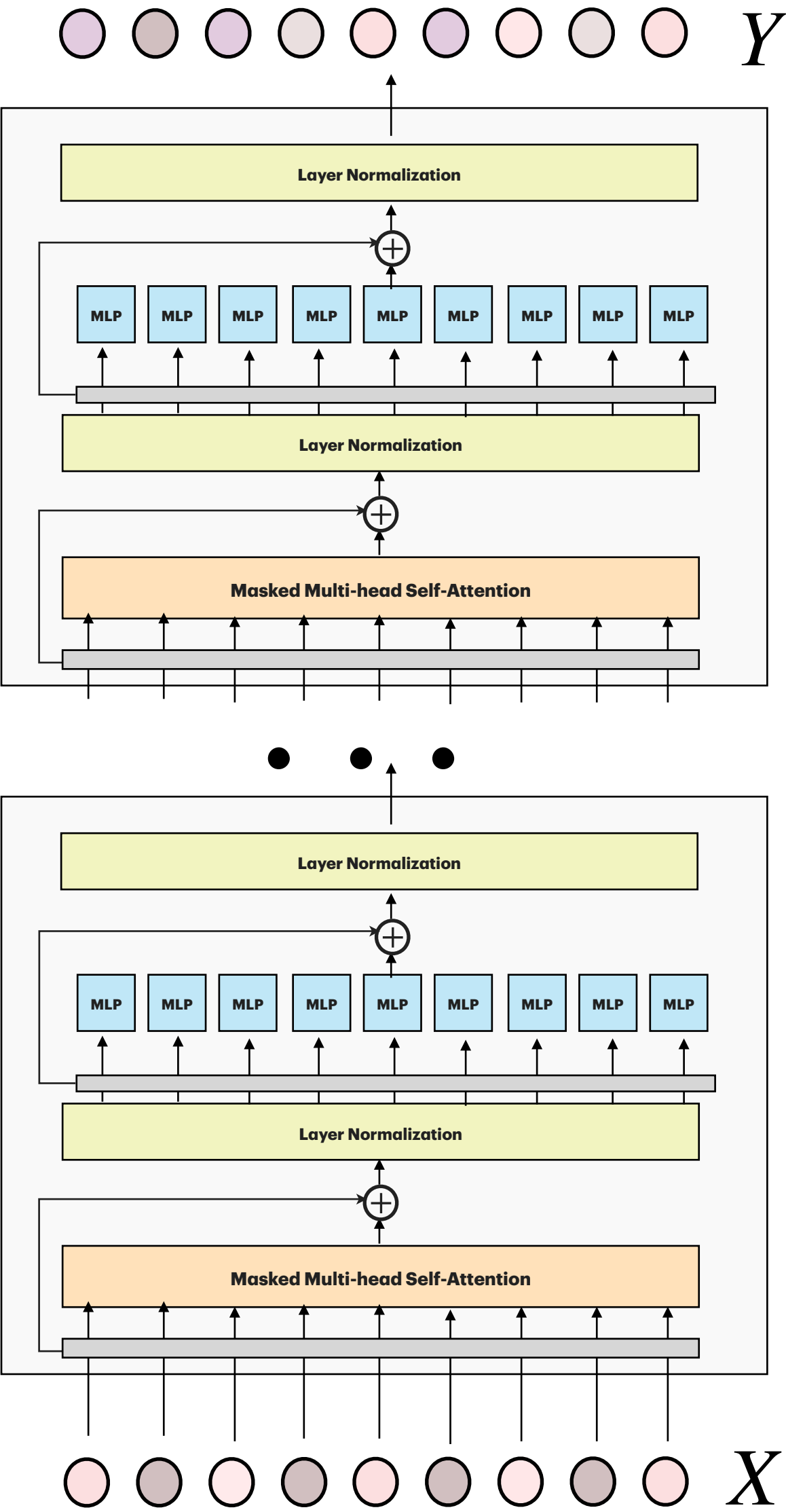
Message passing follows graph edges only

Suggested **Graph Transformer**
computation graph

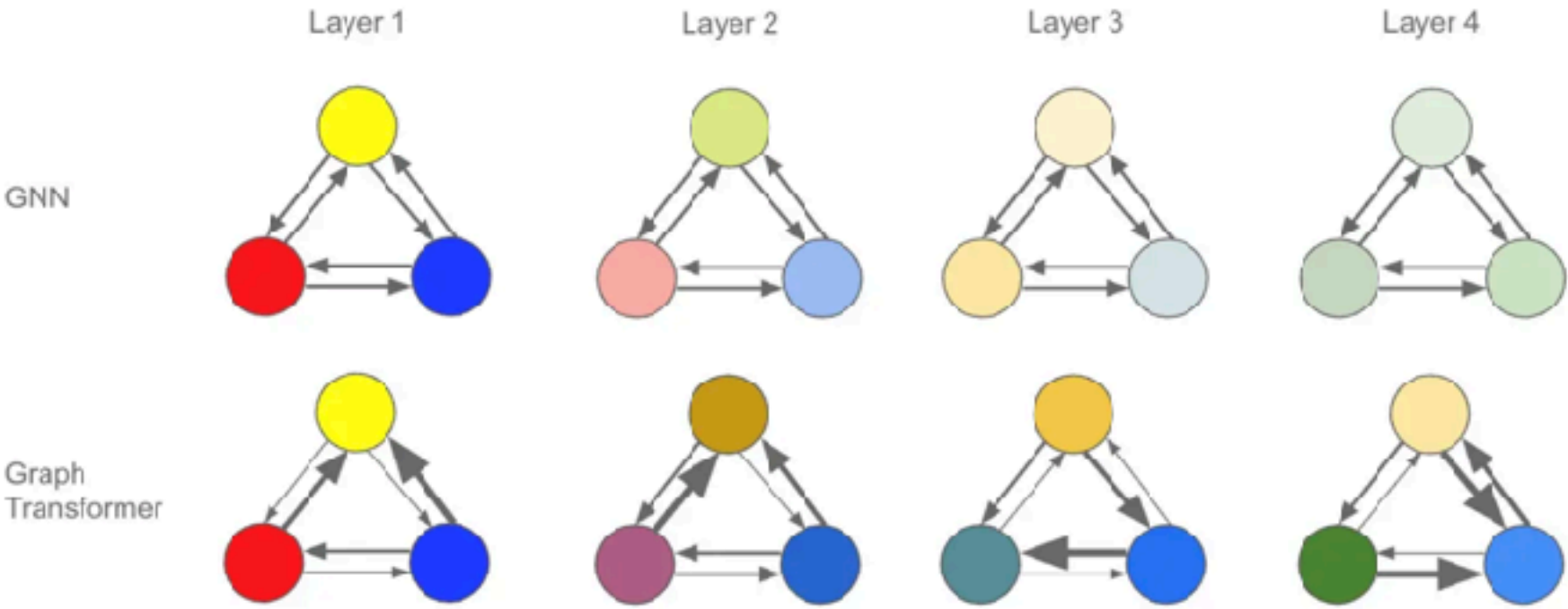
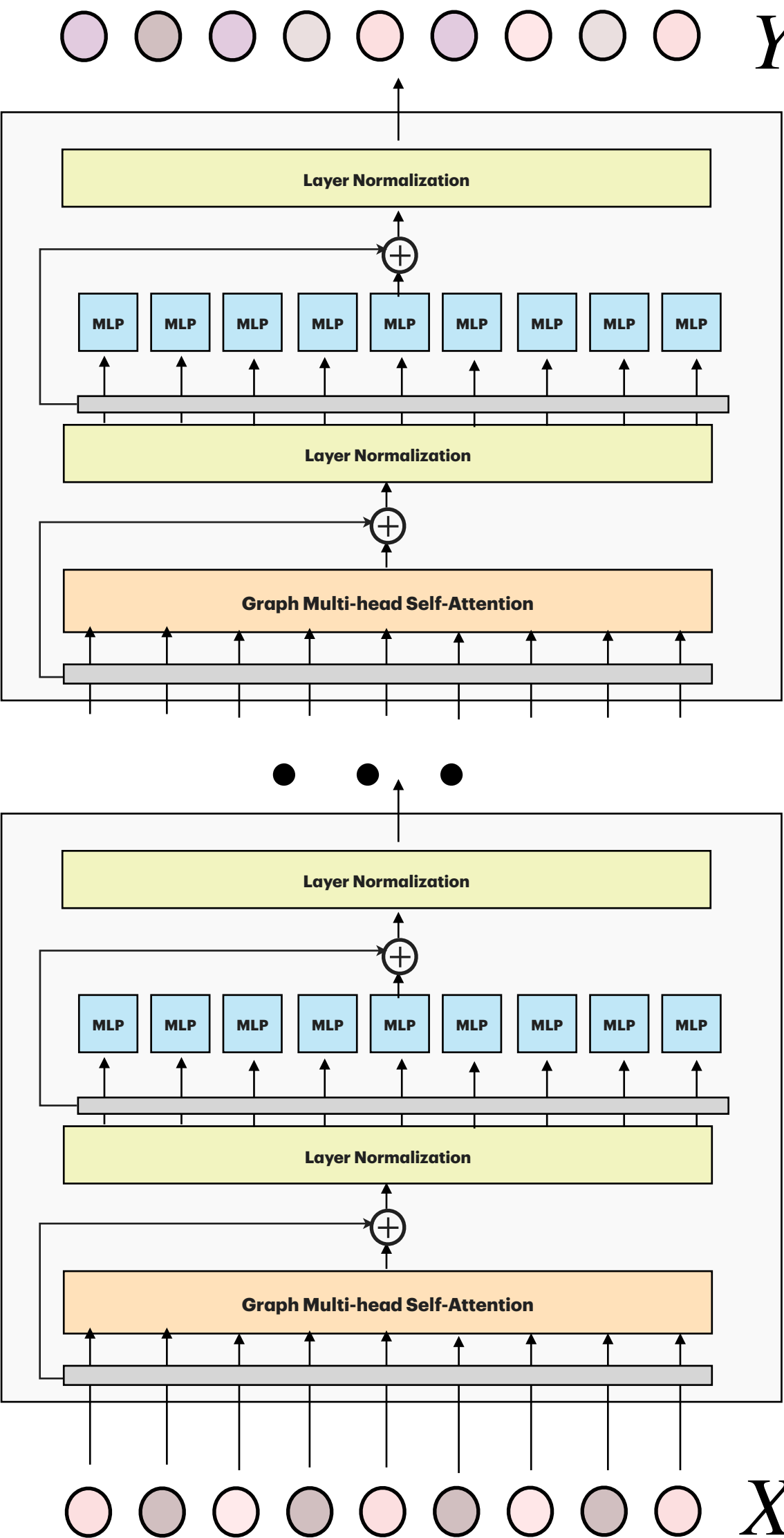


Masked Self-attention follows graph edges only

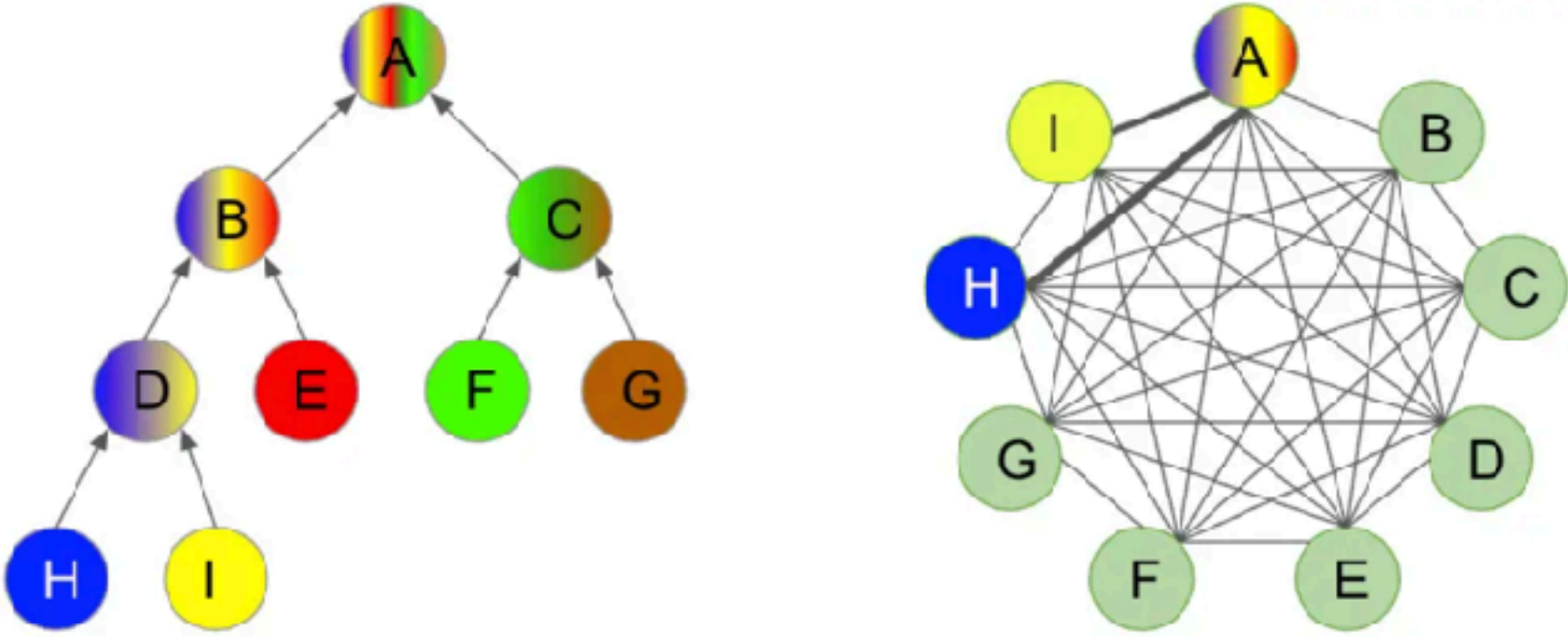
Graph Transformers: Graph Transformer introduces an inductive bias by incorporating the graph's topology into the attention mechanism. Instead of allowing every node to attend to all others



Graph Transformers V2: Another variant will just add biases to the relevant attention scores, offering greater flexibility and long-range modeling at the cost of higher computational complexity and potential loss of inductive bias.



Top: Deep GNNs cause node features to blur, leading to over-smoothing. Bottom: Graph Transformers use varied attention to preserve node uniqueness and relevance.



Left: GNNs compress distant signals through bottlenecks like node A, losing detail. Right: Graph Transformers use global attention, letting A directly access H and I, avoiding over-squashing.

Aspect	Graph Neural Networks	Graph Transformers
Information Flow	Sequential message passing between connected nodes.	Self-attention allows direct aggregation from all nodes.
Long-Range Dependencies	Struggle with distant relationships due to limited hops.	Capture long-range dependencies more effectively.
Over-Smoothing	Node embeddings become too similar with deep layers.	Less prone, as attention allows flexible information mixing.
Over-Squashing	Information gets compressed when aggregating many neighbors.	Mitigated by attention, distributing information more effectively.
Heterogeneous Models	RGCN, HAN, HGT	HGT, Graphormer, HEAT

PE implementations also might be different in GTs.

Graph Transformers V2: Another variant will just add biases to the relevant attention scores, offering greater flexibility and long-range modeling at the cost of higher computational complexity and potential loss of inductive bias.

Aspect	Pros	Cons
Information Flow	Decouples node updates from graph structure, enabling flexible learning.	Loses the strong locality bias that makes GNNs effective on many structured graphs.
Long-Range Dependencies	Can model distant node interactions naturally through self-attention.	Fully connected attention can lead to unnecessary noise in learning.
Graph Structure Awareness	Can incorporate edge features and relational information.	Requires careful design to ensure graph structure is not lost.
Computational Complexity	No sequential message passing bottleneck, allows parallel computation.	Attention mechanism has $O(N^2)$ complexity vs. $O(E)$ for GNNs, making scalability a challenge for large graphs.
Positional Encoding	Can use graph-aware positional encodings to differentiate nodes.	Designing effective positional encodings for graphs remains an active research area.
Heterogeneous Graphs	Handles multiple node and edge types effectively via attention.	More complex to implement than standard GNN architectures for heterogeneous data.
Flexibility	Allows for learned adaptive neighborhood aggregation instead of fixed message passing.	Without strong inductive biases, requires more data to generalize well.

Excellent blog post:

<https://kumo.ai/research/introduction-to-graph-transformers/>

References:

A Generalization of Transformer Networks to Graphs
Vijay Prakash Dwivedi, Xavier Bresson
arXiv:2012.09699

Recipe for a General, Powerful, Scalable Graph Transformer
Ladislav Rampášek, Mikhail Galkin, Vijay Prakash Dwivedi, Anh Tuan Luu, Guy Wolf, Dominique Beaini
arXiv:2205.12454